

|                                       |           |
|---------------------------------------|-----------|
| <b>1 Reminder</b>                     | <b>1</b>  |
| 1.1 Observations and Tricks           | 1         |
| 1.2 Bug List                          | 1         |
| <b>2 Init (Linux)</b>                 | <b>1</b>  |
| 2.1 vimrc                             | 1         |
| 2.2 bashrc                            | 1         |
| <b>3 Basic</b>                        | <b>1</b>  |
| 3.1 Template (Using Codebook)         | 1         |
| 3.2 PBDS, Random                      | 1         |
| 3.3 Debug                             | 1         |
| 3.4 SVG Writer                        | 1         |
| 3.5 Python                            | 2         |
| 3.6 Stress Tests                      | 2         |
| <b>4 Data Structure</b>               | <b>2</b>  |
| 4.1 Mo's Algorithm                    | 2         |
| 4.2 CDQ                               | 2         |
| 4.3 Persistent Treap                  | 2         |
| 4.4 Li Chao Tree                      | 3         |
| 4.5 Time Segment Tree                 | 3         |
| 4.6 Segment Tree Beats                | 3         |
| <b>5 DP</b>                           | <b>4</b>  |
| 5.1 SOS DP                            | 4         |
| 5.2 Divide and Conquer DP             | 4         |
| 5.3 Dynamic DP                        | 4         |
| 5.4 Convex Hull Trick                 | 4         |
| <b>6 Graph</b>                        | <b>4</b>  |
| 6.1 Max Clique                        | 4         |
| 6.2 Bellman-Ford                      | 5         |
| 6.3 System of Difference Constraints  | 5         |
| 6.4 Graph Girth                       | 5         |
| 6.5 Euler Trail                       | 5         |
| 6.6 Vertex BCC (Round Square Tree)    | 5         |
| 6.7 Edge BCC                          | 6         |
| 6.8 Kth Shortest Path                 | 6         |
| 6.9 SCC - Tarjan                      | 6         |
| 6.10 2SAT                             | 7         |
| <b>7 Tree</b>                         | <b>7</b>  |
| 7.1 Tree Isomorphism (Rooted Trees)   | 7         |
| 7.2 Tree Isomorphism (Unrooted Trees) | 7         |
| 7.3 Heavy Light Decomposition         | 7         |
| 7.4 Virtual Tree                      | 7         |
| <b>8 Matching</b>                     | <b>7</b>  |
| 8.1 Bipartite Matching                | 7         |
| 8.2 Bipartite Weighted Matching       | 8         |
| 8.3 General Matching                  | 8         |
| <b>9 Flow</b>                         | <b>8</b>  |
| 9.1 Flow Methods                      | 8         |
| 9.2 Dinic (Flow Decomposition)        | 9         |
| 9.3 ISAP                              | 9         |
| 9.4 Bounded Max Flow                  | 10        |
| 9.5 MCMF                              | 10        |
| 9.6 Push-Relabel                      | 10        |
| 9.7 Gomory-Hu Tree                    | 11        |
| 9.8 Global Min Cut                    | 11        |
| <b>10 String</b>                      | <b>11</b> |
| 10.1 Rolling Hash                     | 11        |
| 10.2 KMP                              | 11        |
| 10.3 Z Value                          | 11        |
| 10.4 Manacher                         | 11        |
| 10.5 Suffix Array + LCP               | 11        |
| 10.6 Suffix Automaton                 | 12        |
| 10.7 Minimum Rotation                 | 12        |

## 1 Reminder

### 1.1 Observations and Tricks

- Contribution Technique
- 二分圖/Spanning Tree/DFS Tree
- 行、列操作互相獨立
- 奇偶性
- 當  $s, t$  遞增並且  $t = f(s)$ ，對  $s$  二分搜不好做，可以改成對  $t$  二分搜，再算  $f(t)$
- 啟發式合併
- Permutation Normalization (做一些平移對齊兩個 permutation)
- 枚舉  $a_1 \sim a_n$  再枚舉  $a_n \sim a_1$  可以包在一個迴圈
- 兩個凸型函數相加還是凸型函數，相減不一定
- 一個區間的  $\text{mex} = k$ ，表示這個區間包含  $0 \sim k-1$  所有數字，並且「 $U$  - 區間」的最小值  $= k$ 。

### 1.2 Bug List

- 沒開 long long
- 陣列戳出界／陣列開不夠大
- 寫好的函式 (例如 `init()`、`build()`) 忘記呼叫
- 0-base / 1-base
- 線段樹改值懶標初始值不能設為 0
- DFS 的時候不小心覆寫到全域變數
- 多筆測資不能沒讀完直接 return
- vector 超級肥，小 vector 請用 array，例如矩陣快速冪

## 2 Init (Linux)

### 2.1 vimrc

```
1 syn on
```

|                                                         |           |
|---------------------------------------------------------|-----------|
| 10.8 AC Automaton                                       | 12        |
| 10.9 AC Automaton - Dict Links                          | 12        |
| 10.10 AC Automaton - Counting Patterns                  | 12        |
| 10.11 AC Automaton - Pattern Position                   | 12        |
| <b>11 Geometry</b>                                      | <b>12</b> |
| 11.1 Template                                           | 12        |
| 11.2 Basic                                              | 13        |
| 11.3 Line / Segment Intersections                       | 13        |
| 11.4 Point-Line / Segment Distance                      | 13        |
| 11.5 Sort                                               | 13        |
| 11.6 Convex Hull                                        | 13        |
| 11.7 Triangle Centers                                   | 13        |
| 11.8 Point Inside                                       | 13        |
| 11.9 Point In Triangle                                  | 14        |
| 11.10 Vector In Polygon                                 | 14        |
| 11.11 Polygon Cut                                       | 14        |
| 11.12 Half-Plane Intersection                           | 14        |
| 11.13 Cyclic Ternary Search                             | 14        |
| 11.14 Circle Intersections                              | 14        |
| 11.15 Tangents                                          | 15        |
| 11.16 Bisector                                          | 15        |
| 11.17 Minimum Enclosing Circle                          | 15        |
| 11.18 Minkowski Sum                                     | 15        |
| 11.19 Convex Hull Distance                              | 15        |
| 11.20 Closest Point Pair                                | 15        |
| 11.21 Polygon-Circle Intersection                       | 15        |
| 11.22 Union of Polygons                                 | 15        |
| 11.23 Rotating Sweep Line                               | 16        |
| 11.24 Pick's Theorem                                    | 16        |
| <b>12 Number Theory</b>                                 | <b>16</b> |
| 12.1 Mod Sum                                            | 16        |
| 12.2 Extended Lucas Theorem                             | 16        |
| 12.3 Prime Sieve and Defactor                           | 17        |
| 12.4 Harmonic Series                                    | 17        |
| 12.5 Count Number of Divisors                           | 17        |
| 12.6 數論分塊                                               | 17        |
| 12.7 Pollard's rho                                      | 17        |
| 12.8 Miller Rabin                                       | 17        |
| 12.9 Discrete Log                                       | 18        |
| 12.10 Discrete Sqrt                                     | 18        |
| 12.11 Fast Power                                        | 18        |
| 12.12 Extend GCD                                        | 18        |
| 12.13 Mu + Phi                                          | 18        |
| 12.14 Other Formulas                                    | 18        |
| 12.15 Polynomial                                        | 19        |
| 12.16 Counting Primes                                   | 20        |
| 12.17 Linear Sieve for Other Number Theoretic Functions | 20        |
| <b>13 Numerical</b>                                     | <b>20</b> |
| 13.1 Polynomials and recurrences                        | 20        |
| 13.2 Optimization                                       | 21        |
| 13.3 Matrices                                           | 22        |
| 13.4 Fourier transforms                                 | 24        |
| <b>14 Combinatorics</b>                                 | <b>25</b> |
| 14.1 Catalan Number                                     | 25        |
| 14.2 Bertrand's Ballot Theorem                          | 25        |
| 14.3 Burnside's Lemma                                   | 25        |
| <b>15 Special Numbers</b>                               | <b>25</b> |
| 15.1 Fibonacci Series                                   | 25        |
| 15.2 Prime Numbers                                      | 25        |
| 15.3 Number of Divisors                                 | 25        |
| 15.4 Distinct Prime Factors                             | 25        |

```
2 set nu rnu cul mouse=a cin et ts=2 sw=2 sts=2 autochdir
3 cb=unnamedplus
4 no ; :
5 ino {<CR> {<CR>}<Esc>ko
6 ca Hash w !cpp -dD -P \ | tr -d '[:space:]' \ | openssl sha256
7 -r \ | cut -c-6
```

```
1 " Gino's .vimrc "
2 "no <C-h> ^ | <C-l> $ | h b | l e | b l | e h
3 }
2.2 bashrc c08ff4
```

```
1 gogo() {
2   g++ -std=c++20 -O2 "$1" && echo run && ./a.out
3 }
4 alias vig='vim -u ~/.vimrc-gino'
```

## 3 Basic

### 3.1 Template (Using Codebook)

```
1 // CRYPTOGRAPHER
2 #define SZ(c) ((int)(c).size())
3 // kactl
4 #define sz(x) (int)(x).size()
5 #define rep(i,a,b) for(int i=(a);i<(b);i++)
6 using ll = long long;
7 using vi = vector<int>;
8 using pii = pair<int, int>;
9 // ckiseki
10 #define all(x) begin(x), end(x)
11 #define pb emplace_back
12 #define REP(i, l, r) for (int i=(l); i<=(r); ++i)
13 const ll LINF = 1e18;
14 template<class A, class B> bool chmin(A& a, B& b) {
15   return b < a ? a = b, 1 : 0;
16 }
```

### 3.2 PBDS, Random

```
1 #include <bits/extc++.h>
2 using namespace __gnu_pbds;
3 // map
4 tree<int, int, less<>, rb_tree_tag,
5   tree_order_statistics_node_update> tr;
6 tr.order_of_key(element);
7 tr.find_by_order(rank);
8 // set
9 tree<int, null_type, less<>, rb_tree_tag,
10   tree_order_statistics_node_update> tr;
11 tr.order_of_key(element);
12 tr.find_by_order(rank);
13 // priority queue
14 __gnu_pbds::priority_queue<int, less<int>> big_q;
15 __gnu_pbds::priority_queue<int, greater<int>> small_q;
16 ql.join(q2); // join
17 mt19937
18 gen(chrono::steady_clock::now().time_since_epoch().count());
19 uniform_int_distribution<int>(a, b)(rng) // [a, b]
20 uniform_real_distribution<double>(a, b)(rng) // (a, b)
21 shuffle(v.begin(), v.end(), gen);
```

### 3.3 Debug

```
1 #define debug(...) cerr << "\x1b[33m #_VA_ARGS_ ":
2   \x1b[0m", \
3   [ ](auto&&... a){ ((cerr << a << ' '), ...); }
4   (_VA_ARGS_), cerr << '\n'
5 #define output(...) \
6   [ ](auto&&... a){ ((cout << a << ' '), ...); }
7   (_VA_ARGS_), cout << '\n'
8 int v = 42;
9 format("{:b}", v); // "101010"
10 format("{:#b}", v); // "0b101010"
11 format("{:x}", v); // "0x2a"
12 format("{:#x}", v); // "0X2A"
13 int x = 7;
14 format("{:05d}", x); // "00007"
15 format("{:>5}", x); // " 7"
16 format("{:<5}", x); // "7 "
17 format("{:^5}", x); // " 7 "
18 format("{:*^5}", x); // "**7**"
19 double pi = 3.1415926535;
20 format("{:.2f}", pi); // "3.14"
```

### 3.4 SVG Writer

```
1 // Author: ckiseki
2 class SVG {
3   void p(string_view s) { o << s; }
4   void p(string_view s, auto v, auto... vs) {
5     auto i = s.find('$');
6     o << s.substr(0, i) << v, p(s.substr(i + 1), vs...);
7   }
8   ofstream o; string c = "red";
9 public:
10   SVG(auto f, auto x1, auto y1, auto x2, auto y2) : o(f) {
11     p("<svg xmlns='http://www.w3.org/2000/svg' "
```

```

12     "viewBox='$ $ $ $>\n"
13     "<style>{*{stroke-width:0.5%;}</style>\n",
14     x1, -y2, x2 - x1, y2 - y1); }
15 ~SVG() { p("</svg>\n"); }
16 void color(string nc) { c = nc; }
17 void line(auto x1, auto y1, auto x2, auto y2) {
18     p("<line x1='$' y1='$' x2='$' y2='$' stroke='$' />\n",
19     x1, -y1, x2, -y2, c); }
20 void circle(auto x, auto y, auto r) {
21     p("<circle cx='$' cy='$' r='$' stroke='$' "
22     "fill='none' />\n", x, -y, r, c); }
23 void text(auto x, auto y, string s, int w = 12) {
24     p("<text x='$' y='$' font-size='$px'></text>\n",
25     x, -y, w, s); }
26 };

```

```

1 SVG svg("out.svg", -200, -200, 200, 200);
2 svg.color("lightgray"); // ↓ drawing x-y plane
3 svg.line(-200, 0, 200, 0); svg.line(0, -200, 0, 200);
4 svg.color("blue"); svg.line(10, 10, 80, 80);
5 svg.color("red"); svg.circle(50, 50, 1);

```

### 3.5 Python

b16508

```

1 import sys
2 input = sys.stdin.readline
3 readStr = lambda: input().rstrip('\r\n')
4 readInt = lambda: int(input())
5 readInts = lambda: list(map(int, input().split()))
6 from decimal import *
7 getcontext().prec = 50 # precision
8 x = Decimal(str(x))
9 y = Decimal("6.7")
10 x *= y
11 print(x.quantize(Decimal("0.000001"),
12     rounding=ROUND_HALF_EVEN))
13 # ROUND_CEILING : To INF (2.9 -> 3, -2.1 -> -2)
14 # ROUND_FLOOR : To -INF (2.1 -> 2, -2.9 -> -3)
15 # ROUND_HALF_EVEN: To nearest even (2.5 -> 2, 3.5 -> 4)
16 # ROUND_HALF_UP : Half away from 0 (2.5 -> 3, -2.5 -> -3)
17 # ROUND_HALF_DOWN: Half towards 0 (2.5 -> 2, -2.5 -> -2)
18 # ROUND_UP : Away from 0 (2.1 -> 3, -2.1 -> -3)
19 # ROUND_DOWN : Truncate to 0 (2.9 -> 2, -2.9 -> -2)
20 # <Default>: ROUND_HALF_EVEN
21 # <Standard School Math>: ROUND_HALF_UP

```

### 3.6 Stress Tests

```

1 from random import *
2 n = randint(1, 10)
3 arr = list(randint(1, 20) for i in range(n))
4 print(*arr)
5 # permutation / non-repetitive sequence
6 perm = sample(range(1, n + 1), n)
7 arr = sample(range(1, 10000 + 1), n)
8 # string
9 from string import *
10 s = "".join(choices(ascii_lowercase, k=n))
11 t = "".join(choices(ascii_lowercase[:3], k=n)) # (a|b|c)*
12 u = "".join(choices(ascii_lowercase + ascii_uppercase +
13     digits, k=n))
14 # palindrome
15 half = "".join(choices(ascii_lowercase, k=(n + 1) // 2))
16 palindrome = half + (half[-2::-1] if n % 2 else half[:-1])
17 # ===== tree =====
18 n = 30
19 ## shallow tree
20 edges = [(randint(1, u - 1), u) for u in range(2, n + 1)]
21 ## deep tree
22 ## (suggested k: 2 ~ 5 or int(sqrt(n)) or 2n/D (D is
23     expected depth))
24 from math import *
25 k = int(sqrt(n))
26 edges = [(randint(max(1, u-k), u-1), u) for u in range(2, n
27     + 1)]
28 for e in edges:
29     print(*e)
30 edges = [(1, i) for i in range(2, n + 1)] ## star
31 edges = [(i, i + 1) for i in range(1, n)] ## chain
32 edges = [(i // 2, i) for i in range(2, n + 1)] ## complete
33     binary tree
34 # ===== graph =====
35 ## not necessarily connected, no multi-edge or self-loop
36 n = 10
37 density = 0.3 # graph density (0.0 ~ 1.0)
38 edges = [(i, j) for i in range(1, n+1) for j in range(i+1,
39     n+1) if random() < density]
40 ## connected, no multi-edge or self-loop
41 n, extra_edges = 10, 5
42 edges = set()
43 ##### ... generate spanning tree with codes above
44 while len(edges) < n - 1 + extra_edges:

```

```

40     u, v = randint(1, n), randint(1, n)
41     if u != v:
42         edges.add((min(u, v), max(u, v)))
43 edges = list(edges)
44 ## DAG
45 perm = sample(range(1, n + 1), n)
46 edges = []
47 for i in range(m):
48     i, j = sorted(sample(range(n), 2))
49     edges.append((perm[i], perm[j]))
50 ## Bipartite Graph
51 n1, n2 = 6, 7
52 left, right = list(range(1, n1+1)), list(range(n1+1,
53     n1+n2+1))
54 edges = [(choice(left), choice(right)) for _ in range(m)]

```

```

1 g++ -std=c++20 -o buggy $1
2 g++ -std=c++20 -o ac ac.cpp
3 for i in {1..100}; do
4     echo "TEST $i"
5     python3 gen.py > in
6     ./ac < in > oa
7     ./buggy < in > ob
8     if ! diff -q oa ob; then
9         cat in <(echo "=== AC") oa <(echo "=== WA") ob
10        break
11    fi
12 done

```

## 4 Data Structure

### 4.1 Mo's Algorithm

8d7194

```

1 // segments are 0-based
2 int B = max(1, (int)sqrt(n));
3 sort(Q.begin(), Q.end(), [&](const auto& a, const auto& b) {
4     if (a[0]/B != b[0]/B) return a[0]/B < b[0]/B;
5     return (a[0]/B & 1) ? a[1] < b[1] : a[1] > b[1];
6 });
7 ll cur = 0; // current answer
8 int pl = 0, pr = -1;
9 for (auto& qi : Q) {
10    // get (l, r, qid) from qi
11    while (pl < l) del(pl++); while (pl > l) add(--pl);
12    while (pr < r) add(++pr); while (pr > r) del(pr--);
13    ans[qid] = cur;
14 }

```

### 4.2 CDQ

744a80

```

1 // sorted by x; type 0 = update (weight w), 1 = query
2 // query: ans[id] = sum of w over updates with x' <= x,
3 // y' <= y, z' <= z (< if strict[dim]). 值域 [1, mxc]
4 // Dependency: 值域 BIT
5 struct Info { int x, y, z, type; ll w; int id; };
6 struct CDQ {
7     static bool cy(const Info& p, const Info& q)
8     { return p.y < q.y; }
9     vector<Info> a; int n; array<bool, 3> strict;
10    vector<ll> ans; BIT bit;
11    void dfs(int l, int r) { if (r - l <= 1) return;
12        int m = (l + r) / 2, i = l;
13        dfs(l, m);
14        vector<Info> t(a.begin() + m, a.begin() + r);
15        ranges::sort(t, cy);
16        for (auto& q : t) {
17            for (; i < m && a[i].y < q.y + !strict[1]; i++)
18                if (!a[i].type) bit.upd(a[i].z, a[i].w);
19            if (q.type)
20                ans[q.id] += bit.qry(q.z - strict[2]);
21        }
22        for (int k = l; k < i; k++)
23            if (!a[k].type) bit.upd(a[k].z, -a[k].w);
24        dfs(m, r);
25        inplace_merge(a.begin() + l, a.begin() + m,
26            a.begin() + r, cy);
27    }
28    CDQ(vector<Info> _a, int mxc, array<bool, 3> _s = {}):
29        a(_a), n(a.size()), strict(_s) {
30        ans.assign(n, 0); bit.init(mxc);
31        auto key = [&](Info& p) { return
32            make_tuple(p.x, p.type != strict[0], p.y, p.z); };
33        ranges::sort(a, {}, key); dfs(0, n);
34    }
35 };

```

### 4.3 Persistent Treap

803cce

```

1 // Author: Ian
2 struct node {
3     node *l, *r;
4     char c; int v, sz;
5     node(char x = '$'): c(x), v(mt()), sz(1) {
6         l = r = nullptr;

```

```

7 }
8 node(node* p) { *this = *p; }
9 void pull() {
10     sz = 1;
11     for (auto i : {l, r})
12         if (i) sz += i->sz;
13 }
14 arr[maxn], *ptr = arr;
15 inline int size(node* p) { return p ? p->sz : 0; }
16 node* merge(node* a, node* b) {
17     if (!a || !b) return a ? a : b;
18     if (a->v < b->v) {
19         node* ret = new(ptr++) node(a);
20         ret->r = merge(ret->r, b); ret->pull();
21         return ret;
22     }
23     else {
24         node* ret = new(ptr++) node(b);
25         ret->l = merge(a, ret->l); ret->pull();
26         return ret;
27     }
28 }
29 P<node*> split(node* p, int k) {
30     if (!p) return {nullptr, nullptr};
31     if (k >= size(p->l) + 1) {
32         auto [a, b] = split(p->r, k - size(p->l) - 1);
33         node* ret = new(ptr++) node(p);
34         ret->r = a; ret->pull();
35         return {ret, b};
36     }
37     else {
38         auto [a, b] = split(p->l, k);
39         node* ret = new(ptr++) node(p);
40         ret->l = b; ret->pull();
41         return {a, ret};
42     }
43 }

```

#### 4.4 Li Chao Tree

5ad3d3

```

1 // Author: Ian
2 // insert(L): add line L(ax+b) in [0, maxn)
3 // insertseg(L, ql, qr): add segment L in [ql, qr)
4 // query(x): find maximum L at x
5 using T=long long;
6 const int maxn = 1e5+5;
7 const T MN = -1e15;
8 struct line {
9     T a, b;
10     line(T a=0, T b=MN): a(a), b(b) {}
11     T operator()(T x) { return a * x + b; }
12 } arr[(maxn+1)<<3];
13 bool operator<(line a, line b) {
14     return a.a < b.a || (a.a == b.a && a.b < b.b); }
15 #define m ((l+r)>>1)
16 void insert(line x, int i=1, int l=0, int r=maxn) {
17     if (r-l == 1) {
18         if (x(l) > arr[i](l)) arr[i] = x;
19         return;
20     }
21     line a = max(arr[i], x), b = min(arr[i], x);
22     if (a(m) > b(m))
23         arr[i] = a, insert(b, i<<1, l, m);
24     else
25         arr[i] = b, insert(a, i<<1|1, m, r);
26 }
27 void insertseg(line x, int ql, int qr,
28     int i=1, int l=0, int r=maxn) {
29     if (qr<=l || r<=ql || ql>=qr) return;
30     if (ql<=l && r<=qr) { insert(x, i, l, r); return; }
31     insertseg(x, ql, qr, i<<1, l, m);
32     insertseg(x, ql, qr, i<<1|1, m, r);
33 }
34 T query(T x, int i=1, int l=0, int r=maxn) {
35     if (x<l || r<=x) return MN;
36     if (r-l == 1) return arr[i](x);
37     return max({arr[i](x),
38         query(x, i<<1, l, m), query(x, i<<1|1, m, r)});
39 }
40 #undef m

```

#### 4.5 Time Segment Tree

519775

```

1 // Author: Ian
2 struct Op; // reversible operation
3 const int maxn = 2e5 + 5;
4 vector<Op> arr[(maxn + 1) << 2];
5 #define m ((l + r) >> 1)
6 void insert(int ql, int qr, Op x, int i=1, int l=0, int r=maxn) {
7     if (qr <= l || r <= ql) return;
8     if (ql <= l && r <= qr) { arr[i].push_back(x); return; }
9     if (qr <= m) insert(ql, qr, x, i << 1, l, m);
10     else if (m <= ql) insert(ql, qr, x, i << 1 | 1, m, r);

```

```

11     else {
12         insert(ql, qr, x, i << 1, l, m);
13         insert(ql, qr, x, i << 1 | 1, m, r);
14     }
15 void traversal(vector<int>& ans, int i=1, int l=0, int r=maxn) {
16     int opcnt = 0;
17     for (auto val : arr[i])
18         if (DO_SOMETHING(val)) opcnt++;
19     if (r - l == 1) ans[l] = CALC_ANS();
20     else {
21         traversal(ans, i << 1, l, m);
22         traversal(ans, i << 1 | 1, m, r);
23     }
24     while (opcnt--) UNDO_SOMETHING();
25     arr[i].clear();
26 }
27 #undef m

```

#### 4.6 Segment Tree Beats

f686aa

```

1 // Author: Ian, https://judge.yosupo.jp/submission/400861
2 const int maxn=2e5+5;
3 const ll MX = 1e18+5;
4 const ll MN = -MX;
5 #define m ((l+r)>>1)
6 struct node {
7     ll sum, inc;
8     ll mn, smn; int mnc;
9     ll mx, smx; int mxc;
10     node(ll x=0): sum(x), inc(0),
11         mn(x), smn(MX), mnc(1),
12         mx(x), smx(MN), mxc(1) {}
13     void pull(int i);
14     void push(int i, int l, int r);
15 } a[(maxn+1)<<2];
16 void node::pull(int i) {
17     sum = a[i<<1].sum+a[i<<1|1].sum;
18     if (a[i<<1].mn == a[i<<1|1].mn) {
19         mn = a[i<<1].mn;
20         smn = min(a[i<<1].smn, a[i<<1|1].smn);
21         mnc = a[i<<1].mnc+a[i<<1|1].mnc;
22     } else {
23         if (a[i<<1].mn < a[i<<1|1].mn) {
24             mn = a[i<<1].mn;
25             smn = min(a[i<<1].smn, a[i<<1|1].mn);
26             mnc = a[i<<1].mnc;
27         } else {
28             mn = a[i<<1|1].mn;
29             smn = min(a[i<<1|1].smn, a[i<<1].mn);
30             mnc = a[i<<1|1].mnc;
31         }
32     }
33     if (a[i<<1].mx == a[i<<1|1].mx) {
34         mx = a[i<<1].mx;
35         smx = max(a[i<<1].smx, a[i<<1|1].smx);
36         mxc = a[i<<1].mxc+a[i<<1|1].mxc;
37     } else {
38         if (a[i<<1].mx > a[i<<1|1].mx) {
39             mx = a[i<<1].mx;
40             smx = max(a[i<<1].smx, a[i<<1|1].mx);
41             mxc = a[i<<1].mxc;
42         } else {
43             mx = a[i<<1|1].mx;
44             smx = max(a[i<<1|1].smx, a[i<<1].mx);
45             mxc = a[i<<1|1].mxc;
46         }
47     }
48 void updmn(int i, int len, ll v) {
49     a[i].sum += (v-a[i].mn)*a[i].mnc; a[i].mn = v;
50     if (len==1 || v>=a[i].mx)
51         a[i].mx = v, a[i].smx = MN;
52     else if (v>a[i].smx)
53         a[i].smx = v;
54 }
55 void updmx(int i, int len, ll v) {
56     a[i].sum -= (a[i].mx-v)*a[i].mxc; a[i].mx = v;
57     if (len==1 || v<=a[i].mn)
58         a[i].mn = v, a[i].smn = MX;
59     else if (v<a[i].smn)
60         a[i].smn = v;
61 }
62 void node::push(int i, int l, int r) {
63     if (r-l==1) return;
64     pair<int, int> ch[] = {{i<<1, m-l}, {i<<1|1, r-m}};
65     if (inc!=0) for (auto [j, len]: ch) {
66         a[j].sum += inc*len, a[j].inc += inc;
67         a[j].mx += inc, a[j].smx += inc;
68         a[j].mn += inc, a[j].smn += inc;
69     }
70     inc = 0;
71     for (auto [j, len]: ch) {
72         if (a[j].mn < mn) updmn(j, len, mn);

```

```

71     if (a[j].mx > mx) updmx(j, len, mx); }
72 }
73 void build(vector<ll>& v, int i=1, int l=0, int r=maxn) {
74     if ((int)v.size() <= l) return;
75     if (r-l==1) { a[i] = node(v[l]); return; }
76     build(v, i<<1, l, m);
77     build(v, i<<1|1, m, r);
78     a[i].pull(i);
79 }
80 void chmin(int ql, int qr, ll v, int i=1, int l=0, int r=maxn) {
81     if (qr<=l || r<=ql || a[i].mx<=v) return;
82     if (ql<=l && r<=qr && a[i].smx<v) { updmx(i, r-l, v); return; }
83     a[i].push(i, l, r);
84     chmin(ql, qr, v, i<<1, l, m);
85     chmin(ql, qr, v, i<<1|1, m, r);
86     a[i].pull(i);
87 }
88 void chmax(int ql, int qr, ll v, int i=1, int l=0, int r=maxn) {
89     if (qr<=l || r<=ql || a[i].mn>=v) return;
90     if (ql<=l && r<=qr && a[i].smn>v) { updmn(i, r-l, v); return; }
91     a[i].push(i, l, r);
92     chmax(ql, qr, v, i<<1, l, m);
93     chmax(ql, qr, v, i<<1|1, m, r);
94     a[i].pull(i);
95 }
96 void inc(int ql, int qr, ll v, int i=1, int l=0, int r=maxn) {
97     if (qr<=l || r<=ql) return;
98     if (ql<=l && r<=qr) {
99         a[i].sum += v*(r-l);
100        a[i].mx += v, a[i].smx += v;
101        a[i].mn += v, a[i].smn += v;
102        a[i].inc += v;
103        return;
104    }
105    a[i].push(i, l, r);
106    inc(ql, qr, v, i<<1, l, m);
107    inc(ql, qr, v, i<<1|1, m, r);
108    a[i].pull(i);
109 }
110 ll qsum(int ql, int qr, int i=1, int l=0, int r=maxn) {
111     if (qr<=l || r<=ql) return 0;
112     if (ql<=l && r<=qr) return a[i].sum;
113     a[i].push(i, l, r);
114     return qsum(ql, qr, i<<1, l, m) + qsum(ql, qr, i<<1|1, m, r);
115 }
116 #undef m

```

## 5 DP

### 5.1 SOS DP

623a08

```

1 // Author: Gino
2 // Function: Solve problems that enumerates subsets of
3 // subsets (3^n => n*2^n)
4 for (int msk = 0; msk < (1<<n); msk++) {
5     for (int i = 1; i <= n; i++) {
6         if (msk & (1<<(i-1))) {
7             dp[msk][i] = dp[msk][i-1] + dp[msk ^ (1<<(i-1))][i-1];
8         } else {
9             dp[msk][i] = dp[msk][i-1];
10        }
11    }
12 }

```

### 5.2 Divide and Conquer DP

d36c77

```

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: Given $a[i] = \min_{lo(i) \le k < hi(i)} (f(i, k))$ where the $[lo(i), hi(i))$
3 * optimal $k$ increases with $i$, computes $a[i]$ for $i = L..R-1$.
4 * Time: $O((N + (hi-lo)) \log N)$ */
5 struct DP { // Modify at will:
6     int lo(int ind) { return 0; }
7     int hi(int ind) { return ind; }
8     ll f(int ind, int k) { return dp[ind][k]; }
9     void store(int ind, int k, ll v) { res[ind] = pii(k, v); }
10    void rec(int L, int R, int LO, int HI) {
11        if (L >= R) return;
12        int mid = (L + R) >> 1;
13        pair<ll, int> best(LLONG_MAX, LO);
14        rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
15            best = min(best, make_pair(f(mid, k), k));
16        store(mid, best.second, best.first);
17        rec(L, mid, LO, best.second+1);
18        rec(mid+1, R, best.second, HI);
19    }
20    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
21 };

```

### 5.3 Dynamic DP

- 適用情境：\$dp\_i = M\_i \cdot dp\_{i-1} \Rightarrow dp\_i = M\_i M\_{i-1} \dots M\_1 dp\_0\$
- 當 \$M\_i\$ 需要動態修改，且 \$M\_i\$ 是廣義矩陣乘法：

$$C_{ij} = \bigoplus_k A_{ik} \otimes B_{kj}$$

- 滿足 \$(\oplus, \otimes)\$ 是半環 (\$\otimes\$ 對 \$\oplus\$ 有分配律、\$\otimes\$ 和 \$\oplus\$ 有結合律、\$\oplus\$ 有交換律)
- 此時便可用線段樹維護 \$M\_n M\_{n-1} \dots M\_1\$ 的乘積，線段樹上的 pos 是 \$M\_{pos}\$，但 pull 時要 \$st[idx] = mul(st[cr], st[cl])\$

## 5.4 Convex Hull Trick

bb4a1c

```

1 /**
2 * Author: Simon Lindholm (License: CC0, Source: own work)
3 * Container where you can add lines of the form $kx+m$,
4 * and query maximum values at points $x$.
5 * Useful for dynamic programming ('convex hull trick').
6 * Time: $O(\log N)$, Status: stress-tested */
7 struct Line {
8     mutable ll k, m, p;
9     bool operator<(const Line& o) const { return k < o.k; }
10    bool operator<(ll x) const { return p < x; }
11 };
12 struct LineContainer : multiset<Line, less<>> {
13     // (for doubles, use inf = 1/.0, div(a,b) = a/b)
14     static const ll inf = LLONG_MAX;
15     ll div(ll a, ll b) { // floored division
16         return a / b - ((a ^ b) < 0 && a % b); }
17     bool isect(iterator x, iterator y) {
18         if (y == end()) return x->p = inf, 0;
19         if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
20         else x->p = div(y->m - x->m, x->k - y->k);
21         return x->p >= y->p;
22     }
23     void add(ll k, ll m) {
24         auto z = insert({k, m, 0}), y = z++, x = y;
25         while (isect(y, z)) z = erase(z);
26         if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
27         while ((y = x) != begin() && (--x)->p >= y->p)
28             isect(x, erase(y));
29     }
30     ll query(ll x) {
31         auto l = *lower_bound(x);
32         return l.k * x + l.m;
33     };
34 };

```

## 6 Graph

### 6.1 Max Clique

3a1fe5

```

1 // max N = 64
2 typedef unsigned long long ll;
3 struct MaxClique {
4     ll nb[ N ], n, ans;
5     void init( ll _n ) {
6         n = _n;
7         for ( int i = 0 ; i < n ; i ++ ) nb[ i ] = 0LLU;
8     }
9     void add_edge( ll _u , ll _v ) {
10        nb[ _u ] |= ( 1LLU << _v );
11        nb[ _v ] |= ( 1LLU << _u );
12    }
13    void B( ll r , ll p , ll x , ll cnt , ll res ) {
14        if ( cnt + res < ans ) return;
15        if ( p == 0LLU && x == 0LLU ) {
16            if ( cnt > ans ) ans = cnt;
17            return;
18        }
19        ll y = p | x; y &= -y;
20        ll q = p & ( ~nb[ int( log2( y ) ) ] );
21        while( q ) {
22            ll i = int( log2( q & (-q) ) );
23            B( r | ( 1LLU << i ) , p & nb[ i ] , x & nb[ i ] ,
24                cnt + 1LLU , __builtin_popcountll( p &
25                    nb[ i ] ) );
26            q &= ~( 1LLU << i );
27            p &= ~( 1LLU << i );
28            x |= ( 1LLU << i );
29        }
30        int solve() {
31            ans = 0;
32            ll _set = 0;
33            if ( n < 64 ) _set = ( 1LLU << n ) - 1;
34            else {
35                for ( ll i = 0 ; i < n ; i ++ ) _set |= ( 1LLU << i );
36            }
37            B( 0LLU , _set , 0LLU , 0LLU , n );
38            return ans;
39        }
40    } maxClique;

```

## 6.2 Bellman-Ford

92b21f

```

1 // Author: Gino
2 // n, m = #(vertices), #(edges), and vertices are 1-based
3 // 1. BellmanFord bf(G, n, m);
4 // 2-1: bf.calcDis(s);
5 // => bf.dis[u] := shortest dis from s to u
6 // => bf.dis[u] := LINF (s can't reach u)
7 // => bf.dis[u] := -LINF (dis[u] can be arbitrary small)
8 // 2-2: bf.findNegCycle();
9 // => false (if no neg cycle)
10 // => true (has neg cycle & gives an example: bf.neg_cycle)
11 // Time: O(VE), Space: O(V + E)
12 const ll LINF = 4e18;
13 struct BellmanFord {
14     const vector<vector<pair<int, ll>>>& G;
15     int n, m; // #(vertices), #(edges)
16     BellmanFord(const auto& G, int n, int m):G(G), n(n), m(m) {}
17     vector<ll> dis; vector<int> nth_relax, pa;
18     void run_bf(const auto& src) {
19         dis.assign(n+1, LINF);
20         nth_relax.assign(n+1, 0); pa.assign(n+1, -1);
21         for (auto& s : src) dis[s] = 0;
22         for (int rlx = 1; rlx <= n; rlx++) {
23             for (int u = 1; u <= n; u++) {
24                 if (dis[u] == LINF) continue; // !important
25                 for (auto& [v, w] : G[u]) {
26                     if (dis[v] > dis[u] + w) {
27                         dis[v] = dis[u] + w; pa[v] = u;
28                         if (rlx == n) nth_relax[v] = 1;
29                     }
30                 }
31             }
32         } // 5 brackets
33     }
34     void calcDis(int s) {
35         run_bf(vector<int>{s}); queue<int> q;
36         for (int u=1; u<=n; u++) if (nth_relax[u]) q.push(u);
37         while (!q.empty()) { int u = q.front(); q.pop();
38             for (auto& [v, w] : G[u]) {
39                 if (!nth_relax[v]) {
40                     nth_relax[v] = 1; q.push(v);
41                 }
42             }
43         }
44         for (int u=1; u<=n; u++) if (nth_relax[u]) dis[u] = -LINF;
45     }
46     vector<int> neg_cycle;
47     bool findNegCycle() {
48         auto src = views::iota(1, n + 1); run_bf(src);
49         auto it = ranges::find_if(src,
50             [&](int s){ return nth_relax[s]; });
51         if (it == src.end()) return false;
52         int ptr = *it; for (int i=0; i<n; i++) ptr = pa[ptr];
53         neg_cycle.clear(); int cur = ptr;
54         while (true) {
55             neg_cycle.emplace_back(cur);
56             if (cur == ptr && neg_cycle.size() > 1) break;
57             cur = pa[cur];
58         }
59         ranges::reverse(neg_cycle); return true;
60     }
61 };

```

## 6.3 System of Difference Constraints

fe55f9

```

1 vector<vector<pair<int, ll>>> G;
2 void add(int u, int v, ll w) { G[u].push_back({v, w}); }

```

- $x_u - x_v \leq c \Rightarrow \text{add}(v, u, c)$
- $x_u - x_v \geq c \Rightarrow \text{add}(u, v, -c)$
- $x_u - x_v = c \Rightarrow \text{add}(v, u, c), \text{add}(u, v, -c)$
- $x_u \geq c \Rightarrow \text{add super vertex } x_0 = 0, \text{ then } x_u - x_0 \geq c \Rightarrow \text{add}(u, 0, -c)$
- Don't forget non-negative constraints for every variable if specified implicitly.
- Interval sum  $\Rightarrow$  Use prefix sum to transform into differential constraints. Don't forget  $S_{i+1} - S_i \geq 0$  if  $x_i$  needs to be non-negative.
- $x_u/x_v \leq c \Rightarrow \log x_u - \log x_v \leq \log c$

## 6.4 Graph Girth

Run BFS for every node, when encountered non-BFS-tree edge, update answer (min cycle length) with  $\text{dis}[u] + \text{dis}[v] + 1$ . Time  $O(VE)$ .

## 6.5 Euler Trail

11c393

```

1 // Author: kactl (Simon Lindholm), wrapped by Gino
2 // Vertices are 1-based, Edge ID are 0-based
3 // 1. EulerWalk euler(n, true(directed) / false(undirected));
4 // 2. euler.addEdge(u, v);
5 // 3. int result = euler.solve();
6 // => -1: nothing (euler.path, euler.path_edges = {})
7 // => 1: Euler trail,
8 // => 2: Euler circuit
9 //     euler.path = {u_1, ..., u_{m+1}}
10 //     euler.path_edges = {e_1, ..., e_m}
11 // Time: O(V + E), Space: O(V + E)

```

```

12 // Test: V, E <= 2e5, 99ms
13 struct EulerWalk {
14     int n, m = 0; bool dir;
15     vector<vector<pii>> G; vi deg;
16     EulerWalk(int n, bool dir) :
17         n(n), dir(dir), G(n+1), deg(n+1) {}
18     void addEdge(int u, int v) {
19         G[u].push_back({v, m});
20         if (!dir) G[v].push_back({u, m});
21         deg[u]++, deg[v] += dir ? -1 : 1; m++;
22     }
23     vi path, path_edges;
24     void walk(int src) {
25         path.clear(); path_edges.clear();
26         vi its(n+1), visE(m);
27         function<void(int, int)> dfs = [&](int u, int e_in) {
28             while (its[u] < sz(G[u])) {
29                 auto [v, e] = G[u][its[u]++];
30                 if (visE[e]) continue;
31                 visE[e] = 1;
32                 dfs(v, e);
33             }
34             path.push_back(u);
35             if (e_in != -1) path_edges.push_back(e_in);
36         };
37         dfs(src, -1);
38         if (sz(path) != m+1) {
39             path.clear(); path_edges.clear(); return;
40         }
41         ranges::reverse(path); ranges::reverse(path_edges);
42     }
43     int solve() {
44         int s = -1, odd = 0, src = 0, dst = 0;
45         for (int u = 1; u <= n; u++) {
46             if (!dir && (deg[u] & 1)) odd++, s = u;
47             if (dir && deg[u] == 1) s = u, src++;
48             if (dir && deg[u] == -1) dst++;
49             if (dir && abs(deg[u]) > 1) return -1;
50         }
51         if (!dir && odd != 0 && odd != 2) return -1;
52         if (dir && !((src == 0 && dst == 0) || (src == 1 && dst == 1))) return -1;
53         if (s == -1) for (int u = 1; u <= n; u++)
54             if (!G[u].empty()) { s = u; break; }
55         if (s == -1) s = 1;
56         walk(s);
57         return path.empty() ? -1 :
58             path.front() == path.back() ? 2 : 1;
59     }
60 };

```

## 6.6 Vertex BCC (Round Square Tree)

e85b0a

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399308
2 // Vertex BCC (Round Square Tree) 0-based; Edge ID 0-based
3 // 1. BCC bcc(n); // n = #(vertices)
4 // 2. bcc.add_edge(u, v);
5 // 3. bcc.build();
6 // => bcc[id]: vertices in bcc 'id' (square node n+id in T)
7 // => T[id]: tree adjacency; square nodes are n..n+ncnt-1
8 // => is_cut(x): whether x is a cut vertex
9 // => bcc_id(u, v): square node containing tree-edge (u, v)
10 struct BCC {
11     int n, m, t, ncnt;
12     vector<vector<pii>> G;
13     vector<vector<int>> T, bcc;
14     vector<int> dep, low, pa2, dep2, s;
15     void add_edge(int u, int v) {
16         G[u].eb(v, m), G[v].eb(u, m++);
17     }
18     void dfs(int x, int f) {
19         if (~dep[x]) return;
20         dep[x] = low[x] = t++;
21         s.push_back(x);
22         for (auto [y, e]: G[x]) if (e != f) {
23             if (dep[y] == -1) {
24                 dfs(y, e), low[x] = min(low[x], low[y]);
25                 if (low[y] >= dep[x]) {
26                     int id = n + ncnt++; p;
27                     bcc.resize(bcc.size()+1);
28                     do {
29                         p = s.back(), s.pop_back();
30                         T[id].push_back(p), T[p].push_back(id);
31                         bcc.back().push_back(p);
32                     } while (p != y);
33                     T[id].push_back(x), T[x].push_back(id);
34                     bcc.back().push_back(x);
35                 }
36             } else low[x] = min(low[x], dep[y]);
37         }
38     }
39     void dfs2(int x, int p) {
40         dep2[x] = ~p ? dep2[p] + 1 : 0, pa2[x] = p;
41         for (auto y: T[x]) if (y != p) dfs2(y, x);
42     }
43 };

```

```

39 bool is_cut(int x) { return (int)T[x].size() != 1; }
40 int bcc_id(int u, int v) {
41     return pa2[dep2[u] < dep2[v] ? v : u] - n; }
42 void build() {
43     for (int i = 0; i < n; i++) if (dep[i] == -1) {
44         if (G[i].empty()) { // isolated vertex: singleton bcc
45             dep[i] = low[i] = t++;
46             int id = n + ncnt++;
47             bcc.push_back({i});
48             T[id].push_back(i), T[i].push_back(id);
49         } else dfs(i, -1);
50         dfs2(i, -1);
51     } }
52 BCC(int n): n(n), m(0), t(0), ncnt(0), G(n), T(2*n),
53     dep(n, -1), low(n), pa2(2*n), dep2(2*n) {}
54 };

```

## 6.7 Edge BCC

26729d

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399295
2 // Edge BCC, 0-based; Edge ID 0-based
3 // 1. EBCC ebcc(n); // n = #(vertices)
4 // 2. ebcc.add_edge(u, v);
5 // 3. ebcc.build();
6 // => bcc[i]: each edge bcc
7 // => bccn[x]: in which bcc is x
8 // => isbr[e]: whether edge e is a bridge
9 struct EBCC {
10     int n, m, t, ncnt;
11     vector<vector<pii>> G;
12     vector<vector<int>> bcc;
13     vector<bool> isbr;
14     vector<int> dep, low, bccn, s;
15     EBCC(int n): n(n), m(0), t(0), ncnt(0), G(n),
16         dep(n, -1), low(n), bccn(n) {}
17     void add_edge(int u, int v) {
18         G[u].eb(v, m), G[v].eb(u, m++); }
19     void dfs(int x, int f) {
20         if (~dep[x]) return;
21         dep[x] = low[x] = t++;
22         s.push_back(x);
23         for (auto [y, e]: G[x]) if (e != f) {
24             if (dep[y] == -1)
25                 dfs(y, e), low[x] = min(low[x], low[y]);
26             else
27                 low[x] = min(low[x], dep[y]);
28         }
29         if (dep[x] == low[x]) {
30             if (~f) isbr[f] = true;
31             bcc.resize(bcc.size()+1);
32             int p; do {
33                 bccn[p = s.back()] = ncnt;
34                 s.pop_back();
35                 bcc.back().push_back(p);
36             } while (p != x); ncnt++;
37         }
38     void build() {
39         isbr.assign(m, false);
40         for (int i=0; i<n; i++) dfs(i, -1); }
41 };

```

## 6.8 Kth Shortest Path

34bacd

```

1 // time: O(|E| lg |E|+|V| lg |V|+K)
2 // memory: O(|E| lg |E|+|V|)
3 struct KSP { // 1-base
4     struct nd {
5         int u, v; ll d;
6         nd(int ui=0, int vi=0, ll di=INF) { u=ui; v=vi; d=di; }
7     };
8     struct heap { nd* edge; int dep; heap* chd[4]; };
9     static int cmp(heap* a, heap* b)
10     { return a->edge->d > b->edge->d; }
11     struct node {
12         int v; ll d; heap* H; nd* E;
13         node() {}
14         node(ll _d, int _v, nd* _E) { d=_d; v=_v; E=_E; }
15         node(heap* _H, ll _d) { H=_H; d=_d; }
16         friend bool operator<(node a, node b)
17         { return a.d > b.d; }
18     };
19     int n, k, s, t, dst[N]; nd *nxt[N];
20     vector<nd*> g[N], rg[N]; heap *nullNd, *head[N];
21     void init(int _n, int _k, int _s, int _t) {
22         n=_n; k=_k; s=_s; t=_t;
23         for (int i=1; i<=n; i++) {
24             g[i].clear(); rg[i].clear();
25             nxt[i]=NULL; head[i]=NULL; dst[i]=-1;
26         }
27     void addEdge(int ui, int vi, ll di) {
28         nd* e=new nd(ui, vi, di);
29         g[ui].push_back(e); rg[vi].push_back(e);

```

```

30     }
31     queue<int> dfsQ;
32     void dijkstra() {
33         while (dfsQ.size()) dfsQ.pop();
34         priority_queue<node> Q; Q.push(node(0, t, NULL));
35         while (!Q.empty()) {
36             node p=Q.top(); Q.pop(); if (dst[p.v] != -1) continue;
37             dst[p.v]=p.d; nxt[p.v]=p.E; dfsQ.push(p.v);
38             for (auto e: rg[p.v]) Q.push(node(p.d+e->d, e->u, e));
39         }
40         heap* merge(heap* curNd, heap* newNd) {
41             if (curNd==nullNd) return newNd;
42             heap* root=new heap; memcpy(root, curNd, sizeof(heap));
43             if (newNd->edge->d < curNd->edge->d) {
44                 root->edge=newNd->edge;
45                 root->chd[2]=newNd->chd[2];
46                 root->chd[3]=newNd->chd[3];
47                 newNd->edge=curNd->edge;
48                 newNd->chd[2]=curNd->chd[2];
49                 newNd->chd[3]=curNd->chd[3];
50             }
51             if (root->chd[0]->dep < root->chd[1]->dep)
52                 root->chd[0]=merge(root->chd[0], newNd);
53             else root->chd[1]=merge(root->chd[1], newNd);
54             root->dep=max(root->chd[0]->dep,
55                 root->chd[1]->dep)+1;
56             return root;
57         }
58         vector<heap*> V;
59         void build() {
60             nullNd=new heap; nullNd->dep=0; nullNd->edge=new nd;
61             fill(nullNd->chd, nullNd->chd+4, nullNd);
62             while (not dfsQ.empty()) {
63                 int u=dfsQ.front(); dfsQ.pop();
64                 if (!nxt[u]) head[u]=nullNd;
65                 else head[u]=head[nxt[u]->v];
66                 V.clear();
67                 for (auto& e: g[u]) {
68                     int v=e->v;
69                     if (dst[v] == -1) continue;
70                     e->d+=dst[v]-dst[u];
71                     if (nxt[u] != e) {
72                         heap* p=new heap; fill(p->chd, p->chd+4, nullNd);
73                         p->dep=1; p->edge=e; V.push_back(p);
74                     }
75                 }
76                 if (V.empty()) continue;
77                 make_heap(V.begin(), V.end(), cmp);
78                 #define L(X) ((X<<1)+1)
79                 #define R(X) ((X<<1)+2)
80                 for (size_t i=0; i<V.size(); i++) {
81                     if (L(i)<V.size()) V[i]->chd[2]=V[L(i)];
82                     else V[i]->chd[2]=nullNd;
83                     if (R(i)<V.size()) V[i]->chd[3]=V[R(i)];
84                     else V[i]->chd[3]=nullNd;
85                 }
86                 head[u]=merge(head[u], V.front());
87             }
88         vector<ll> ans;
89         void first_K() {
90             ans.clear(); priority_queue<node> Q;
91             if (dst[s] == -1) return;
92             ans.push_back(dst[s]);
93             if (head[s] != nullNd)
94                 Q.push(node(head[s], dst[s]+head[s]->edge->d));
95             for (int _=1; _<k and not Q.empty(); _++) {
96                 node p=Q.top(); Q.pop(); ans.push_back(p.d);
97                 if (head[p.H->edge->v] != nullNd) {
98                     q.H=head[p.H->edge->v]; q.d=p.d+q.H->edge->d;
99                     Q.push(q);
100                 }
101             }
102             for (int i=0; i<4; i++)
103                 if (p.H->chd[i] != nullNd) {
104                     q.H=p.H->chd[i];
105                     q.d=p.d-p.H->edge->d+p.H->chd[i]->edge->d;
106                     Q.push(q);
107                 }
108         void solve() { // ans[i] stores the i-th shortest path
109             dijkstra(); build();
110             first_K(); // ans.size() might less than k
111         }
112     } solver;

```

## 6.9 SCC - Tarjan

7a063d

```

1 // Author: Ian, https://judge.yosupo.jp/submission/399291
2 // Tarjan SCC, 0-based
3 // 1. SCC scc(n); // n = #(vertices)
4 // 2. Fill in adj list G
5 // 3. scc.build()

```

```

6// => scc[i]: each scc
7// => sccn[x]: in which scc is x
8struct SCC {
9    const int inf = 1e9+5;
10   int n, t, ncnt;
11   vector<vector<int>> G, scc;
12   vector<bool> ins;
13   vector<int> sccn, dfn, low, s;
14   SCC(int n): n(n), t(0), ncnt(0), G(n),
15       ins(n, false), sccn(n), dfn(n, -1), low(n, inf) {}
16   void dfs(int x) {
17       if (~dfn[x]) return;
18       dfn[x] = low[x] = t++;
19       s.push_back(x), ins[x] = true;
20       for (auto i: G[x])
21           if (dfs(i), ins[i])
22               low[x] = min(low[x], low[i]);
23       if (dfn[x] == low[x]) {
24           scc.resize(scc.size()+1);
25           int p; do {
26               ins[p = s.back()] = false;
27               s.pop_back(); sccn[p] = ncnt;
28               scc.back().push_back(p);
29           } while (p != x); ncnt++;
30       }
31   }
32   void build() { for (int i=0; i<n; i++) dfs(i); }
33};

```

## 6.10 2SAT

130351

```

1// Author: Ian, CRYPTOGRAPHER
2struct TwoSAT: SCC {
3    TwoSAT(int n): SCC(2*n) {}
4    // (var a == na) v (var b == nb)
5    void add_disjunction(int a, int na, int b, int nb) {
6        a = 2*a^na, b = 2*b^nb;
7        G[a^1].push_back(b);
8        G[b^1].push_back(a);
9    }
10   vector<int> solve() {
11       build(); vector<int> assignment(n/2, 0);
12       for(int i=0; i<n/2; i++) {
13           if (sccn[2*i] == sccn[2*i^1]) return {};
14           assignment[i] = sccn[2*i] > sccn[2*i^1];
15       } return assignment;
16   };

```

## 7 Tree

### 7.1 Tree Isomorphism (Rooted Trees)

f3f029

```

1// Author: Gino
2// Checks if 2 rooted trees are isomorphic
3// Time: O(V log V)
4// vector<vector<int>> G1, G2;
5// int root_1, root_2;
6map<vector<int>, int> dict;
7int encode(const auto& G, int u, int pa = -1) {
8    vector<int> codes;
9    for (auto& v : G[u])
10        if (v != pa)
11            codes.emplace_back(encode(G, v, u));
12    ranges::sort(codes);
13    auto [it, _] = dict.try_emplace(codes, dict.size());
14    return it->second;
15}
16bool isomorphic() {
17    dict.clear(); dict[{}] = 0;
18    return encode(G1, root_1) == encode(G2, root_2);
19}

```

### 7.2 Tree Isomorphism (Unrooted Trees)

Find the centroid(s) of  $T_1, T_2$ .

Case 1:  $T_1, T_2$  have different number of centroids  $\rightarrow$  NO

Case 2:  $T_1$  has centroid  $c_1, T_2$  has centroid  $c_2$

$\rightarrow$  rooted\_isomorphic( $c_1, c_2$ )

Case 3:  $T_1$  has centroids  $c_1, c'_1, T_2$  has centroids  $c_2, c'_2$

$\rightarrow$  rooted\_isomorphic( $c_1, c_2$ ) || rooted\_isomorphic( $c'_1, c'_2$ )

### 7.3 Heavy Light Decomposition

53e45c

```

1// Author: Ian, https://judge.yosupo.jp/submission/399317
2void segbuild(vector<int>&v);
3void segmodify(int p, int k);
4int segquery(int ql, int qr);
5// Insert [ql, qr] segment tree here
6template<class T=int> struct HLD {
7    int n, cnt;
8    vector<int> d, f, sz, son, top, dfn;
9    HLD(int n): n(n), cnt(0), d(n, 0), f(n, 0), sz(n, 1),
10        son(n, -1), top(n, 0), dfn(n, -1) {}
11    void dfs1(int x, int p) {

```

```

12     for (auto i: e[x]) if (i != p) {
13         d[i] = d[x]+1, f[i] = x;
14         dfs1(i, x), sz[x] += sz[i];
15         if (son[x] == -1 || sz[son[x]] < sz[i])
16             son[x] = i;
17     }
18     void dfs2(int x, int t) {
19         dfn[x] = cnt++, top[x] = t;
20         if (son[x] == -1) return;
21         dfs2(son[x], t);
22         for (auto i: e[x])
23             if (!dfn[i]) dfs2(i, i);
24     }
25     void build(const vector<T> &v) {
26         dfs1(0, 0); dfs2(0, 0);
27         vector<T> dfnv(v.size());
28         for (int i=0; i<n; i++)
29             dfnv[dfn[i]] = v[i];
30         segbuild(dfnv); }
31     void modify(int p, T v) { segmodify(dfn[p], v); }
32     T query(int a, int b) {
33         auto agr = [](int x, int y) { return max(x, y); };
34         T ret = 0;
35         while (top[a] != top[b]) {
36             if (d[top[a]] > d[top[b]]) swap(a, b);
37             ret = agr(ret, segquery(dfn[top[b]], dfn[b]+1));
38             b = f[top[b]];
39         }
40         if (dfn[a] > dfn[b]) swap(a, b);
41         return agr(ret, segquery(dfn[a], dfn[b]+1));
42     }
43};

```

## 7.4 Virtual Tree

172166

```

1// Author: std_abs
2// need lca, in, out
3vector<pii> virtual_tree(vector<int> &v) {
4    auto cmp = [&](int x, int y) { return in[x] < in[y]; };
5    sort(all(v), cmp);
6    for (int i = 1; i < sz(v); ++i)
7        v.pb(lca(v[i-1], v[i]));
8    sort(all(v), cmp);
9    v.resize(unique(all(v)) - v.begin());
10   vector<int> stk(1, v[0]);
11   vector<pii> res;
12   for (int i = 1; i < sz(v); ++i) {
13       int x = v[i];
14       while (out[stk.back()] < out[x]) stk.pop_back();
15       res.emplace_back(stk.back(), x), stk.pb(x);
16   }
17   return res;
18}

```

## 8 Matching

### 8.1 Bipartite Matching

f42921

```

1// Author: Gino
2// Function: Max Bipartite Matching in O(V sqrt(E))
3//> hk.init(nx, ny) -> hk.add(x, y (+nx))
4//> hk.max_matching(): the matching plan stores in mx, my
5//> hk.min_vertex_cover(): vertex cover plan stores in vcover
6// (!) vertices are 0-based: X = [0, nx), Y = [nx, nx+ny)
7#define pb emplace_back
8struct HopcroftKarp {
9    int n, nx, ny; vector<vector<int>> G; vector<int> mx, my;
10   HopcroftKarp(int nx, int ny): nx(nx), ny(ny), n(nx + ny)
11       { G.assign(n, vector<int>()); }
12   void add(int x, int y) { G[x].pb(y); G[y].pb(x); }
13   int max_matching() {
14       vector<int> dis, vis;
15       mx.assign(n, -1); my.assign(n, -1);
16       function<bool(int)> dfs = [&](int x) {
17           vis[x] = 1;
18           for (int y: G[x]) { int p = my[y];
19               if (p == -1 ||
20                   (dis[p] == dis[x] + 1 && !vis[p] && dfs(p)))
21                   return mx[x] = y, my[y] = x, true;
22           }
23           return false;
24       };
25       while (true) {
26           dis.assign(n, -1);
27           queue<int> q;
28           for (int x = 0; x < nx; x++)
29               if (mx[x] == -1) dis[x] = 0, q.push(x);
30           while (!q.empty()) {
31               int x = q.front(); q.pop();
32               for (int y: G[x])
33                   if (my[y] != -1 && dis[my[y]] == -1)
34                       dis[my[y]] = dis[x] + 1, q.push(my[y]);

```

```

35 }
36 vis.assign(n, 0);
37 bool found = false;
38 for (int x = 0; x < nx; x++)
39     if (mx[x] == -1 && dfs(x)) found = true;
40 if (!found) break;
41 }
42 int ans = 0;
43 for (int x = 0; x < nx; x++) if (mx[x] != -1) ans++;
44 return ans;
45 }
46 vector<int> vcover;
47 int min_vertex_cover() {
48     int ans = max_matching(); vector<int> vis(n, 0);
49     function<void(int)> dfs = [&](int x) {
50         vis[x] = true;
51         for (int y : G[x]) {
52             if (y == mx[x] || my[y] == -1 || vis[y]) continue;
53             vis[y] = true;
54             dfs(my[y]);
55         };
56         for (int x = 0; x < nx; x++) if (mx[x] == -1) dfs(x);
57         vcover.clear();
58         for (int x=0; x < nx; x++) if (!vis[x]) vcover.pb(x);
59         for (int y=nx; y < nx+ny; y++) if (vis[y]) vcover.pb(y);
60         return ans;
61     };

```

## 8.2 Bipartite Weighted Matching

5933aa

```

1// Author: ckiseki, n = #(left vertices) = #(right vertices)
2// left, right vertices labeled 1 ~ n separately
3// 1. build 1-based matrix G of size (n + 1) * (n + 1)
4// G[u][v] := weight between (left u, right v)
5// <!-- if not need perfect matching
6// ==> set weights of all negative edges to 0
7// 2. construct (solve at same time): KM km(n, G);
8// ==> km.ans (max perfect weight sum)
9// ==> km.fl[u] (left u -> right v)
10// ==> km.fr[v] (right v -> left u)
11// Time: O(V^3), Space: O(V^2), Test: V <= 500, 243ms
12 struct KM { // maximize, test @ UOJ 80
13     int n, l, r; ll ans; // fl and fr are the match
14     vector<ll> hl, hr; vector<int> fl, fr, pre, q;
15     void bfs(const auto &w, int s) {
16         vector<int> vl(n + 1), vr(n + 1);
17         vector<ll> slk(n + 1, LINF);
18         l = r = 0; vr[q[r++] = s] = true;
19         auto check = [&](int x) -> bool {
20             if (vl[x] || slk[x] > 0) return true;
21             vl[x] = true; slk[x] = LINF;
22             if (fl[x] != -1) return (vr[q[r++] = fl[x]] = true);
23             while (x != -1) swap(x, fr[fl[x] = pre[x]]);
24             return false;
25         };
26         while (true) {
27             while (l < r)
28                 for (int x = 1, y = q[l++]; x <= n; ++x) if (!vl[x])
29                     if (ll val = hl[x] + hr[y] - w[x][y]; val < slk[x]) {
30                         slk[x] = val;
31                         if (pre[x] = y, !check(x)) return;
32                     }
33             ll d = LINF;
34             for (int x = 1; x <= n; ++x) d = min(d, slk[x]);
35             for (int x = 1; x <= n; ++x)
36                 vl[x] ? hl[x] += d : slk[x] -= d;
37             for (int x = 1; x <= n; ++x) if (vr[x]) hr[x] -= d;
38             for (int x = 1; x <= n; ++x) if (!check(x)) return;
39         }
40     }
41     KM(int n_, const auto &w) : n(n_), ans(0),
42         hl(n+1), hr(n+1), fl(n+1, -1), fr(fl), pre(n+1), q(n+1) {
43         for (int i = 1; i <= n; ++i) {
44             hl[i] = -LINF;
45             for (int j=1; j <= n; ++j) hl[i] = max(hl[i], (ll)w[i][j]);
46         }
47         for (int i = 1; i <= n; ++i) bfs(w, i);
48         for (int i = 1; i <= n; ++i) ans += w[i][fl[i]];
49     }
50 };

```

## 8.3 General Matching

93f948

```

1// Author: ckiseki
2// 0. build 1-based graph vector<vector<int>> G(n + 1);
3// 1. construct (solve at same time):
4// GeneralMatching M(G, n); ==> M.ans
5// status: M.match[x] (== -1 if not matched, 1 <= x <= n)
6// Time: O(V^3), Space: O(V + E), Test: V <= 500, 11ms
7 struct GeneralMatching {
8     queue<int> q; int ans, n;
9     vector<int> fa, s, v, pre, match;
10    int Find(int u) {

```

```

11    return u == fa[u] ? u : fa[u] = Find(fa[u]); }
12    int LCA(int x, int y) {
13        static int tk = 0; tk++; x = Find(x); y = Find(y);
14        for (; swap(x, y) if (x != 0) {
15            if (v[x] == tk) return x;
16            v[x] = tk;
17            x = Find(pre[match[x]]);
18        } }
19    void Blossom(int x, int y, int l) {
20        for (; Find(x) != l; x = pre[y]) {
21            pre[x] = y, y = match[x];
22            if (s[y] == 1) q.push(y), s[y] = 0;
23            for (int z : {x, y}) if (fa[z] == z) fa[z] = l;
24        } }
25    bool Bfs(auto &&g, int r) {
26        iota(all(fa), 0); ranges::fill(s, -1);
27        q = queue<int>(); q.push(r); s[r] = 0;
28        for (; !q.empty(); q.pop()) {
29            for (int x = q.front(); int u : g[x])
30                if (s[u] == -1) {
31                    if (pre[u] = x, s[u] = 1, match[u] == 0) {
32                        for (int a = u, b = x, last;
33                             b != 0; a = last, b = pre[a])
34                            last = match[b], match[b] = a, match[a] = b;
35                        return true;
36                    }
37                    q.push(match[u]); s[match[u]] = 0;
38                } else if (!s[u] && Find(u) != Find(x)) {
39                    int l = LCA(u, x);
40                    Blossom(x, u, l); Blossom(u, x, l);
41                }
42            }
43        return false;
44    }
45    GeneralMatching(auto &&g, int n_) : ans(0), n(n_),
46        fa(n+1), s(n+1), v(n+1), pre(n+1, 0), match(n+1, 0) {
47        for (int x = 1; x <= n; ++x)
48            if (match[x] == 0) ans += Bfs(g, x);
49        for (int x = 1; x <= n; ++x)
50            if (match[x] == 0) match[x] = -1;
51    };

```

## 9 Flow

### 9.1 Flow Methods

9516b6

```

1// Author: CryptoGrapheR (some modified by Gino)
2Maximize c^T x subject to Ax ≤ b, x ≥ 0;
3with the corresponding symmetric dual problem,
4Minimize b^T y subject to A^T y ≥ c, y ≥ 0.
5Maximize c^T x subject to Ax ≤ b;
6with the corresponding asymmetric dual problem,
7Minimize b^T y subject to A^T y = c, y ≥ 0.
8Maximize \sum x subject to x_i + x_j ≤ A_{ij}, x ≥ 0;
9=> Maximize \sum x subject to x_i + x_j ≤ A_{ij};
10=> Minimize A^T y = \sum A_{ij} y_{ij} subject to for all v,
    \sum_{i=v or j=v} y_{ij} = 1, y_{ij} ≥ 0
11=> possible optimal solution: y_{ij} = {0, 0.5, 1}
12=> y'=2y: \sum_{i=v or j=v} y'_{ij} = 2, y'_{ij} = {0, 1, 2}
13=> Minimum Bipartite perfect matching/2 (V1=X, V2=X, E=A)
14General Graph:
15|Max Ind. Set| + |Min Vertex Cover| = |V|
16|Max Ind. Edge Set| + |Min Edge Cover| = |E|
17Bipartite Graph:
18|Max Ind. Set| = |Min Edge Cover|
19|Max Ind. Edge Set| = |Min Vertex Cover|
20To reconstruct the minimum vertex cover, dfs from each
21unmatched vertex on the left side and with unused edges
22only. Equivalently, dfs from source with unused edges
23only and without visiting sink. Then, a vertex is
24chosen iff. it is on the left side and without visited
25or on the right side and visited through dfs.
26Minimum Weighted Bipartite Edge Cover:
27Construct new bipartite graph with n+m vertices on each
    side:
28for each vertex u, duplicate a vertex u' on the other side
29for each edge (u,v,w), add edges (u,v,w) and (v',u',w)
30for each vertex u, add edge (u,u', 2w) where w is min edge
    connects to u
31then the answer is the minimum perfect matching of the new
    graph (KM)
32Maximum density subgraph ( \sum{W_e} + \sum{W_v} ) / |V|
33Binary search on answer:
34For a fixed D, construct a Max flow model as follow:
35Let S be Sum of all weight( or inf)
361. from source to each node with cap = S
372. For each (u,v,w) in E, (u->v, cap=w), (v->u, cap=w)
383. For each node v, from v to sink with cap = S + 2 * D -
    deg[v] - 2 * (W of v)
39where deg[v] = \sum weight of edge associated with v
40If maxflow < S * |V|, D is an answer.

```

41 Requiring subgraph: all vertex can be reached from source with  
 42 edge whose cap > 0.  
 43 Maximum closed subgraph  
 44 1. connect source with positive weighted vertex(capacity=weight)  
 45 2. connect sink with negative weighted vertex(capacity=-weight)  
 46 3. make capacity of the original edges = inf  
 47 4. ans = sum(positive weighted vertex weight) - (max flow)  
 48 (Node-disjoint) Min DAG Path Cover (用最少路徑覆蓋所有點)  
 49 Node disjoint: 拆出來的路徑不能共用同一個點  
 50 將一個點 u 裂成 u+ 和 u- , 代表進入和出去  
 51 對於一條邊 (u, v) 建邊 u- => v+  
 52 1. +點們和 -點們形成一張二分圖  
 53 2. 最差的答案是 n , 代表每個點自己一個點就是一條路徑  
 54 3. 只要一組 (u-, v+) 匹配成功那對應到的答案恰好會 -1  
 55 >>> ans = n - 這張二分圖的最大匹配  
 56 General DAG Path Cover  
 57 跟 Node-disjoint 版本差在點可以被很多條路共用  
 58 建邊方式 Node-disjoint 差別在條件比較鬆  
 59 Node-disjoint: u- => v+ 只能在 (u, v) 有邊時建  
 60 General: u- => v+ 在 u 能走到 v 的時候就建  
 61 Dilworth Theorem  
 62 反鏈: 一些節點的集合, 滿足這些節點互相無法抵達  
 63 最大反鏈 = 最小 General DAG Path Cover  
 64 Min Cut reconstruction (after running max flow s -> t):  
 65 From s, BFS/DFS through residual edges with remaining  
 66 capacity > 0, marking reached vertices as visited.  
 67 The cut set is every original edge (u, v) with u visited  
 68 and v not visited; its capacity sum equals the max flow.  
 69 Bipartite Matching reconstruction (after running max flow  
 70 on source -> X -> Y -> sink model):  
 71 For each edge (x, y) between the X-side and Y-side, it is  
 72 part of the matching iff. its residual capacity is 0  
 73 (i.e. it is saturated).  
 74 Max/Min Flow with Lower Bound (Circulation Problem):  
 75 1. Construct super source S and super sink T.  
 76 2. For each edge (x, y, l, u), connect x -> y with capacity  
 77 u - l.  
 78 3. For each vertex v, let in(v) = (sum of incoming lower  
 79 bounds)  
 80 - (sum of outgoing lower bounds).  
 81 4. If in(v) > 0, connect S -> v with capacity in(v);  
 82 otherwise connect v -> T with capacity -in(v).  
 83 To maximize: connect t -> s with capacity inf (skip this  
 84 edge  
 85 for a pure circulation problem), let f = maxflow(S, T).  
 86 If  
 87 f != sum of in(v) over all in(v) > 0, no solution exists.  
 88 Otherwise maxflow(s, t) (found after adding t -> s) is  
 89 the answer.  
 90 To minimize: let f = maxflow(S, T) (without any s-t edge  
 yet).  
 91 Then connect t -> s with capacity inf and let f' be the  
 92 extra  
 93 flow pushed from S to T. If f + f' != sum of in(v) over  
 94 all  
 95 in(v) > 0, no solution exists. Otherwise f' is the  
 96 answer.  
 97 5. The flow on each original edge e is l\_e + f\_e, where f\_e  
 98 is  
 99 e's flow in the constructed graph.

## 9.2 Dinic (Flow Decomposition)

b8e775

```
1// Function: Max Flow + Flow Decomposition, O(V^2 E)
2// init -> addEdge; vertices 0-based
3// >>> flow() := return max flow
4// >>> flow_decomposition() := return max flow + store
  decomposition in flow.D
5struct Dinic{
6    static const int MXN=10000;
7    struct Edge{ int v,c,r,C; bool fw; };
8    int n,s,t,level[MXN]; vector<Edge> G[MXN];
9    void init(int _n,int _s,int _t){
10        n=_n; s=_s; t=_t;
11        for (int i=0; i<n; i++) G[i].clear();
12    }
13    void addEdge(int u,int v,int c){
14        G[u].push_back({v,c,(int)G[v].size(),c,true});
15        G[v].push_back({u,0,(int)G[u].size()-1,0,false});
16    }
17    bool BFS(){
18        for (int i=0; i<n; i++) level[i]=-1;
19        queue<int> que; que.push(s); level[s]=0;
20        while(!que.empty()){
21            int u=que.front(); que.pop();
22            for (auto &it:G[u]){
23                if(it.c>0&&level[it.v]==-1){
24                    level[it.v]=level[u]+1; que.push(it.v);
25                }
26            }
27        }
28        return level[t]!=-1;
29    }
```

```
25 int DFS(int u,int nf){
26     if(u==t) return nf;
27     int res=0;
28     for (auto &it:G[u]){
29         if(it.c>0&&level[it.v]==level[u]+1){
30             int tf=DFS(it.v,min(nf,it.c));
31             res+=tf; nf-=tf; it.c-=tf;
32             G[it.v][it.r].c+=tf;
33             if(nf==0) return res;
34         }
35     }
36     if(!res) level[u]=-1;
37     return res;
38 }
39 int flow(int res=0){
40     while(BFS()) res+=DFS(s,2147483647);
41     return res;
42 } // reset: do nothing
43 vector<pair<int, vector<int>>> D; // (flow amount, [path
  p1 ... pk])
44 int flow_decomposition(){
45     int mxflow = flow();
46     vector<vector<Edge>> fG(n); // graph with forward edges
47     for (int u = 0; u < n; u++) {
48         for (auto &e : G[u]) {
49             if (e.fw && e.c - e.c > 0)
50                 fG[u].push_back({e.v, e.C - e.c, e.r, e.C, e.fw});
51         }
52     }
53     vector<int> vis(n, 0);
54     function<int(int, int)> dfs = [&](int u, int cur) {
55         if (u == t) {
56             D.back().second.push_back(u);
57             return cur;
58         }
59         vis[u] = 1;
60         for (auto &e : fG[u]) {
61             if (e.c > 0 && !vis[e.v]) {
62                 int ans = dfs(e.v, min(cur, e.c));
63                 if (ans > 0) {
64                     e.c -= ans;
65                     D.back().second.push_back(u);
66                     return ans;
67                 }
68             }
69         }
70         return 0;
71     };
72     D.clear();
73     int quota = mxflow;
74     while (quota > 0) {
75         D.emplace_back(0, vector<int>());
76         fill(vis.begin(), vis.end(), 0);
77         int f = dfs(s, 2147483647);
78         if (f == 0) break;
79         reverse(D.back().second.begin(),
80                D.back().second.end());
81         D.back().first = f, quota -= f;
82     }
83     return mxflow;
84 }
```

## 9.3 ISAP

c13d55

```
1// Author: CRYPTOGRAPHER
2// 1-based: vertices are numbered 1 ~ n
3// 1. flow.init(n, s, t);
4// n = #vertices, s, t = source, sink
5// 2. flow.addEdge(u, v, c); // u, v in [1, n]
6// 3. call: int ans = flow.flow();
7// => ans = max flow value from s to t
8// Time: O(V^2 * E) worst case, usually much faster in
  practice
9static const int MAXV=50010;
10static const int INF =1000000;
11struct Maxflow{
12    struct Edge{
13        int v,c,r;
14        Edge(int _v,int _c,int _r):v(_v),c(_c),r(_r){}
15    };
16    int s,t; vector<Edge> G[MAXV];
17    int iter[MAXV],d[MAXV],gap[MAXV],tot;
18    void init(int n,int _s,int _t){
19        tot=n,s=_s,t=_t;
20        for(int i=1;i<=tot;i++){
21            G[i].clear(); iter[i]=d[i]=gap[i]=0;
22        }
23    }
24    void addEdge(int u,int v,int c){
25        G[u].push_back(Edge(v,c,SZ(G[v])));
26        G[v].push_back(Edge(u,0,SZ(G[u])-1));
27    }
28    int DFS(int p,int flow){
29        if(p==t) return flow;
30        for(int &i=iter[p]; i<SZ(G[p]); i++){
31            Edge &e=G[p][i];
32            if(e.c>0&&d[p]==d[e.v]+1){
```

```

33     int f=DFS(e.v,min(flow.e.c));
34     if(f){ e.c-=f; G[e.v][e.r].c+=f; return f; }
35 }
36 }
37 if(--gap[d[p]]==0) d[s]=tot;
38 else{ d[p]++; iter[p]=0; ++gap[d[p]]; }
39 return 0;
40 }
41 int flow(){
42     int res=0;
43     for(res=0,gap[0]=tot;d[s]<tot;res+=DFS(s,INF));
44     return res;
45 }
46 void reset(){
47     for(int i=1;i<=tot;i++) iter[i]=d[i]=gap[i]=0;
48 }
49 } flow;

```

#### 9.4 Bounded Max Flow

aaf8c3

```

1// Author: CRYPTOGRAPHER
2// Max flow with lower/upper bound on edges (source-sink
  version)
3// <!-- 1-based: original vertices are numbered 1 ~ n.
4//      Super source, sink: n+1, n+2 respectively.
5// <!-- dependency: ISAP.cpp
6// 1. n = #vertices (1 ~ n), m = #edges
7//      s, t = original source, sink (both in [1, n])
8// 2. l[i], r[i] := edge i goes from l[i] -> r[i] (l[i],
9//      r[i] in [1, n])
10// 3. a[i], b[i] := flow on edge i must lie in [a[i], b[i]]
11// 4. int ans = solve(n, m, s, t);
12// => ans == -1 : no feasible flow exists
13// => ans : feasible max flow value
14// <!-- N, M must be large enough for original graph + 2
   super nodes
15// Time: O(V^2 E), Space: O(V + E)
16int in[N], out[N], l[M], r[M], a[M], b[M];
17int solve(int n, int m, int s, int t){
18    flow.init(n+2, n+1, n+2); // super source = n+1, super
   sink = n+2
19    for(int i = 0; i < m; i++){
20        in[r[i]] += a[i]; out[l[i]] += a[i];
21        flow.addEdge(l[i], r[i], b[i] - a[i]);
22        // flow from l[i] to r[i] must lie in [a[i], b[i]]
23    }
24    int nd = 0;
25    for(int i = 1; i <= n; i++){
26        if(in[i] < out[i]){
27            flow.addEdge(i, flow.t, out[i] - in[i]);
28            nd += out[i] - in[i];
29        }
30        if(out[i] < in[i])
31            flow.addEdge(flow.s, i, in[i] - out[i]);
32    }
33    flow.addEdge(t, s, INF); // original sink -> source
34    if(flow.flow() != nd) return -1; // infeasible
35    int ans = flow.G[s].back().c; // t->s edge's current flow
   = feasible flow s->t
36    flow.G[s].back().c = flow.G[t].back().c = 0;
37    for(size_t i = 0; i < flow.G[flow.s].size(); i++){
38        Maxflow::Edge &e = flow.G[flow.s][i];
39        flow.G[flow.s][i].c = 0; flow.G[e.v][e.r].c = 0;
40    }
41    for(size_t i = 0; i < flow.G[flow.t].size(); i++){
42        Maxflow::Edge &e = flow.G[flow.t][i];
43        flow.G[flow.t][i].c = 0; flow.G[e.v][e.r].c = 0;
44    }
45    flow.addEdge(flow.s, s, INF); flow.addEdge(t, flow.t,
   INF);
46    flow.reset(); // keeps G[] intact, only resets search
   state
47    return ans + flow.flow();
48 }

```

#### 9.5 MCMF

472be6

```

1// Author: CRYPTOGRAPHER
2// MCMF (supports negative edges without negative cycles)
3// <!-- 1-based: vertices are numbered 1 ~ n.
4// 1. MCMF.init(n); // n = #vertices (1 ~ n)
5// 2. MCMF.add(u, v, cap, cost);
6// 3. auto [max_flow, min_cost] = MCMF.flow(s, t); // s, t
   in [1, n]
7// Time: O(F * E), Space: O(V + E)
8typedef int Tcost;
9const int MAXV = 20010;
10const int INFf = 1000000;
11const Tcost INFc = 1e9;
12struct MCMF {
13    struct Edge {
14        int v, cap;
15        Tcost w;

```

```

16    int rev;
17    bool fw;
18    Edge() {}
19    Edge(int t2, int t3, Tcost t4, int t5, bool t6)
   : v(t2), cap(t3), w(t4), rev(t5), fw(t6) {}
20 };
21 int V, s, t;
22 vector<Edge> G[MAXV];
23 void init(int n){
24     V = n;
25     for(int i = 0; i <= V; i++) G[i].clear();
26 }
27 void add(int a, int b, int cap, Tcost w){
28     G[a].push_back(Edge(b, cap, w, (int)G[b].size(), true));
29     G[b].push_back(Edge(a, 0, -w, (int)G[a].size()-1,
   false));
30 }
31 Tcost d[MAXV];
32 int id[MAXV], mom[MAXV];
33 bool inqu[MAXV];
34 queue<int> q;
35 pair<int, Tcost> flow(int _s, int _t){
36     s = _s, t = _t;
37     int mxf = 0, Tcost mnc = 0;
38     while(1){
39         fill(d, d+1+V, INFc); // need to use type cast
40         fill(inqu, inqu+1+V, 0);
41         fill(mom, mom+1+V, -1);
42         mom[s] = s;
43         d[s] = 0;
44         q.push(s); inqu[s] = 1;
45         while(q.size()){
46             int u = q.front(); q.pop();
47             inqu[u] = 0;
48             for(int i = 0; i < (int) G[u].size(); i++){
49                 Edge &e = G[u][i];
50                 int v = e.v;
51                 if(e.cap > 0 && d[v] > d[u]+e.w){
52                     d[v] = d[u]+e.w;
53                     mom[v] = u;
54                     id[v] = i;
55                     if(!inqu[v]) q.push(v), inqu[v] = 1;
56                 }
57             }
58         }
59         if(mom[t] == -1) break;
60         int df = INFf;
61         for(int u = t; u != s; u = mom[u])
62             df = min(df, G[mom[u]][id[u]].cap);
63         for(int u = t; u != s; u = mom[u]){
64             Edge &e = G[mom[u]][id[u]];
65             e.cap -= df;
66             G[e.v][e.rev].cap += df;
67         }
68         mxf += df;
69         mnc += df*d[t];
70     }
71     return make_pair(mxf, mnc);
72 }
73 }
74 };

```

#### 9.6 Push-Relabel

fae237

```

1// Author: Simon Lindholm (kactl)
2// 0-based: vertices are numbered 0 ~ n-1
3// 1. PushRelabel pr(n); // n = #vertices
4// 2. pr.addEdge(u, v, c); // u, v in [0, n-1]
5// 3. ll ans = pr.calc(s, t);
6// => ans = max flow value from s to t
7// To obtain actual flow:
8// for (int u = 0; u < n; u++)
9//     for (auto& e : pr.g[u])
10//         if (e.f > 0) => (u, e.dest, e.f)
11// Time: O(V^2 * sqrt(E)), better on dense graph
12struct PushRelabel {
13    struct Edge { int dest, back; ll f, c; };
14    vector<vector<Edge>> g;
15    vector<ll> ec;
16    vector<Edge*> cur;
17    vector<vi> hs; vi H;
18    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}
19    void addEdge(int s, int t, ll cap, ll rcap=0) {
20        if (s == t) return;
21        g[s].push_back({t, sz(g[t]), 0, cap});
22        g[t].push_back({s, sz(g[s])-1, 0, rcap});
23    }
24    void addFlow(Edge& e, ll f) {
25        Edge &back = g[e.dest][e.back];
26        if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
27        e.f += f; e.c -= f; ec[e.dest] += f;

```

```

28 back.f -= f; back.c += f; ec[back.dest] -= f;
29 }
30 ll calc(int s, int t) {
31     int v = sz(g); H[s] = v; ec[t] = 1;
32     vi co(2*v); co[0] = v-1;
33     rep(i,0,v) cur[i] = g[i].data();
34     for (Edge& e : g[s]) addFlow(e, e.c);
35     for (int hi = 0;;) {
36         while (hs[hi].empty()) if (!hi--) return -ec[s];
37         int u = hs[hi].back(); hs[hi].pop_back();
38         while (ec[u] > 0) // discharge u
39             if (cur[u] == g[u].data() + sz(g[u])) {
40                 H[u] = 1e9;
41                 for (Edge& e : g[u]) if (e.c && H[u] >
H[e.dest]+1)
42                     H[u] = H[e.dest]+1, cur[u] = &e;
43                 if (++co[H[u]], !--co[hi] && hi < v)
44                     rep(i,0,v) if (hi < H[i] && H[i] < v)
45                         --co[H[i]], H[i] = v + 1;
46                 hi = H[u];
47             } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
48                 addFlow(*cur[u], min(ec[u], cur[u]->c));
49             else ++cur[u];
50     }
51 }
52 bool leftOfMinCut(int a) { return H[a] >= sz(g); }
53 }

```

## 9.7 Gomory-Hu Tree

88b014

```

1 // Author: chilli, Takanori MAEHARA (kactl)
2 // <!-- Dependency: PushRelabel.cpp
3 // Given a list of edges representing an undirected flow
graph,
4 // returns edges of the Gomory-Hu tree.
5 // maxflow / mincut of (u, v) => min edge from u to v on
Gomory-Hu tree
6 // Time: O(V) * O(PushRelabel)
7 typedef array<ll, 3> Edge; // (u, v, w)
8 vector<Edge> gomoryHu(int N, vector<Edge> ed) {
9     vector<Edge> tree;
10    vi par(N);
11    rep(i,1,N) {
12        PushRelabel D(N); // Dinic also works
13        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
14        tree.push_back({i, par[i], D.calc(i, par[i])});
15        rep(j,i+1,N)
16            if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
17    }
18    return tree;
19 }

```

## 9.8 Global Min Cut

a040ee

```

1 // Author: ckiseki
2 // 1-based: vertices are numbered 1 ~ n
3 // 1. vector<vector<ll>> G(n+1, vector<ll>(n+1, 0));
4 // 2. add edge(G, u, v, c); // add undirected edge u-v with
weight c, u,v in [1,n]
5 // 3. ll ans = mincut(G, n); // n = #vertices
6 // => ans = global min cut value
7 // Time: O(V^3)
8 void add_edge(auto &w, int u, int v, int c) {
9     w[u][v] += c; w[v][u] += c; }
10 auto phase(const auto &w, int n, vector<int> id) {
11     vector<ll> g(n+1); int s = -1, t = -1;
12     while (!id.empty()) {
13         int c = -1;
14         for (int i : id) if (c == -1 || g[i] > g[c]) c = i;
15         s = t; t = c;
16         id.erase(ranges::find(id, c));
17         for (int i : id) g[i] += w[c][i];
18     } return tuple{s, t, g[t]};
19 }
20 ll mincut(auto w, int n) {
21     ll cut = numeric_limits<ll>::max();
22     vector<int> id(n); iota(all(id), 1);
23     for (int i = 0; i < n - 1; ++i) {
24         auto [s, t, gt] = phase(w, n, id);
25         id.erase(ranges::find(id, t));
26         cut = min(cut, gt);
27         for (int j = 1; j <= n; ++j)
28             w[s][j] += w[t][j], w[j][s] += w[j][t];
29     } return cut;
30 }

```

## 10 String

### 10.1 Rolling Hash

c15659

```

1 // Author: ckiseki (Modified by Gino)
2 // Usage: RH<Sigma, Mod> Hs(s);
3 // If no modifications needed, replace BIT with prefix sum
4 template <ll P = 127, ll Q = 1051762951>

```

```

5 class RH {
6     int n; vector<ll> p, inv, d, bit;
7     ll add(ll a, ll b) { return a+b >= Q ? a+b-Q : a+b; }
8     ll sub(ll a, ll b) { return a-b < 0 ? a-b+Q : a-b; }
9     ll mul(ll a, ll b) { return (__int128)a * b % Q; }
10    ll pw(ll a, ll x) {
11        ll r = 1; for (; x; a = mul(a, a), x >>= 1)
12            if (x & 1) r = mul(r, a);
13        return r;
14    }
15    void bit_add(int i, ll x) {
16        for (; i <= n; i += i&-i) bit[i] = add(bit[i], x); }
17    ll bit_qry(int i) {
18        ll s = 0; for (; i; i -= i&-i) s = add(s, bit[i]);
19        return s; }
20 public:
21     RH(const auto &s) : n(s.size()),
22         p(n+1, 1), inv(n+1, 1), d(n+1), bit(n+1) {
23         ll iv = pw(P, Q - 2);
24         for (int i = 1; i <= n; i++) {
25             p[i] = mul(p[i-1], P); inv[i] = mul(inv[i-1], iv);
26             d[i] = mul(s[i-1], p[i]); bit_add(i, d[i]);
27         }
28         void update(int i, char c) {
29             ll x = mul(c, p[i]); bit_add(i, sub(x, d[i])); d[i] = x; }
30         ll query(int l, int r) {
31             return mul(sub(bit_qry(r), bit_qry(l-1)), inv[l]); }
32 };

```

## 10.2 KMP

376cdf

```

1 // Author: Gino
2 vector<int> kmp(auto s) { int n=s.size(); vector<int> f(n,0);
3     for (int i = 1, ptr = 0; i < n; i++) {
4         while (ptr && s[i] != s[ptr]) ptr = f[ptr-1];
5         if (s[i] == s[ptr]) ptr++; f[i] = ptr;
6     } return f;
7 }

```

## 10.3 Z Value

d5b3b9

```

1 // Author: std_abs
2 vector<int> Z(auto s) { int n=s.size(); vector<int> Z(n);
3     for (int i = 0, l = 0, r = 0; i < n; ++i) {
4         Z[i] = max(min(Z[i-l], r-i), 0);
5         while (i+Z[i] < n && s[Z[i]] == s[i+Z[i]])
6             l = i, r = i+Z[i], Z[i]++;
7     } return Z;
8 }

```

## 10.4 Manacher

193b08

```

1 // Author: std_abs
2 // return array Z (length == 2N - 1)
3 // Example: s = "aaaa"
4 // a a a a
5 // Z_0 Z_1 Z_2 Z_3 Z_4 Z_5 Z_6
6 // 1 2 3 4 3 2 1
7 // The corresponding palindrome s[l..r]:
8 // l = (i - Z[i] + 1) / 2
9 // r = (i + Z[i] - 1) / 2
10 vector<int> manacher(auto tmp) {
11     vector<int> s(1, -1);
12     for (auto c : tmp) s.push_back(c), s.push_back(-1);
13     int l = 0, r = 0, n = (int)s.size();
14     vector<int> Z(n);
15     for (int i = 0; i < n; ++i) {
16         Z[i] = r > i ? min(Z[2 * l - i], r - i) : 1;
17         while (i - Z[i] >= 0 && i + Z[i] < n &&
18             s[i - Z[i]] == s[i + Z[i]]) Z[i]++;
19         if (Z[i] + i > r) l = i, r = Z[i] + i;
20     }
21     for (int i = 0; i < n; ++i)
22         Z[i] = (Z[i] - (i & 1)) / 2 * 2 + (i & 1);
23     return vector<int>(Z.begin() + 1, Z.end() - 1);
24 }

```

## 10.5 Suffix Array + LCP

c1df5b

```

1 // Author: std_abs
2 // Tested @yosupo, N <= 5e5, 123ms
3 const int N = 5e5 + 10;
4 int sa[N], tmp[2][N], c[N], rk[N], lcp[N];
5 void buildSA(string s) {
6     int *x = tmp[0], *y = tmp[1], m = 256, n = s.size();
7     for (int i = 0; i < m; ++i) c[i] = 0;
8     for (int i = 0; i < n; ++i) c[x[i]] = s[i]++;
9     for (int i = 1; i < m; ++i) c[i] += c[i-1];
10    for (int i = n-1; ~i; --i) sa[--c[x[i]]] = i;
11    for (int k = 1; k < n; k <= 1) {
12        for (int i = 0; i < m; ++i) c[i] = 0;
13        for (int i = 0; i < n; ++i) c[x[i]]++;
14        for (int i = 1; i < m; ++i) c[i] += c[i-1];
15        int p = 0;

```

```

16 for (int i = n - k; i < n; ++i) y[p++] = i;
17 for (int i = 0; i < n; ++i) if (sa[i] >= k)
18   y[p++] = sa[i] - k;
19 for (int i = n - 1; ~i; --i)
20   sa[--c[x[y[i]]]] = y[i];
21 y[sa[0]] = p = 0;
22 for (int i = 1; i < n; ++i) {
23   int a = sa[i], b = sa[i - 1];
24   p += !(x[a] == x[b] && a + k < n && b + k < n && x[a + k] == x[b + k]);
25   y[sa[i]] = p;
26 }
27 if (n == p + 1) break;
28 swap(x, y), m = p + 1;
29 } }
30 void buildLCP(string s) {
31   // lcp[i] = LCP(sa[i - 1], sa[i])
32   // those substrs' length in range [len(link) + 1, len]
33   int n = s.length(), x = 0;
34   for (int i = 0; i < n; ++i) rk[sa[i]] = i;
35   for (int i = 0; i < n; ++i) {
36     if (!rk[i]) lcp[rk[i]] = 0;
37     else { if (x) x--;
38            int p = sa[rk[i] - 1];
39            while (x + i < n && x + p < n && s[x + i] == s[x + p]) x++;
40            lcp[rk[i]] = x;
41     } } }

```

## 10.6 Suffix Automaton

10f751

```

1 // Author: std_abs
2 // state (i.e. node) -> all substr with the same endpos
3 // those substrs' length in range [len(link) + 1, len]
4 // state i's endpos set -> union of pos[i] over the subtree
  of i in the suffix link tree
5 // link[i] -> longest suffix with different endpos set
  (suffix link tree edge)
6 // len[i] -> length longest suffix of state i
7 // pos[i] -> an example end position of state i (1-based)
8 // cnt[i] -> size of endpos set of state i, built by
  build_cnt() (tree DP)
9 const int N = 5e5 + 10, C = 26;
10 struct SAM {
11   int ch[2*N][C], len[2*N], link[2*N], pos[2*N], cnt[2*N], _id;
12   SAM () { reset(); }
13   void reset() { _id = 0; newnode(), link[0] = -1; }
14   int newnode() {
15     fill_n(ch[_id], C, 0);
16     len[_id] = link[_id] = pos[_id] = cnt[_id] = 0;
17     return _id++;
18   }
19   void build(string s) {
20     int last = 0; for (int i = 0; i < (int)s.size(); i++) {
21       char c = s[i]; int cur = newnode();
22       len[cur] = len[last] + 1, pos[cur] = i + 1;
23       int p = last;
24       while (~p && !ch[p][c - 'a'])
25         ch[p][c - 'a'] = cur, p = link[p];
26       if (p == -1) link[cur] = 0;
27       else { // conflict: p --> q, p --> cur
28         int q = ch[p][c - 'a'];
29         if (len[p] + 1 == len[q]) { // q is suffix of cur
30           link[cur] = q;
31         } else { // we need auxiliary state
32           int nxt = newnode();
33           len[nxt] = len[p] + 1, link[nxt] = link[q];
34           pos[nxt] = 0;
35           for (int j = 0; j < C; j++)
36             ch[nxt][j] = ch[q][j];
37           while (~p && ch[p][c - 'a'] == q)
38             ch[p][c - 'a'] = nxt, p = link[p];
39           link[q] = link[cur] = nxt;
40         } }
41       cnt[cur]++, last = cur;
42     } }
43 } sam;
44 // void build_cnt() {
45 //   vector<int> p(_id); iota(all(p), 0); sort(all(p),
46 //     [&](int i, int j){ return len[i] > len[j]; });
47 //   for (int i = 0; i < _id; ++i) if (link[p[i]] != -1)
48 //     cnt[link[p[i]]] += cnt[p[i]];
49 // }
50 // bool match(string s) {
51 //   int p = 0; for (auto& c : s) {
52 //     if (!ch[p][c - 'a']) return false;
53 //     p = ch[p][c - 'a'];
54 //   } return true; }

```

## 10.7 Minimum Rotation

bb235f

```

1 // Author: std_abs
2 string rotate(auto s) { int n=s.size(), i=0, j=1;
3   string t = s + s;

```

```

4   while (i < n && j < n) {
5     int k = 0; while (k < n && t[i + k] == t[j + k]) ++k;
6     if (t[i + k] <= t[j + k]) j += k + 1; else i += k + 1;
7     if (i == j) ++j;
8   } return t.substr((i < n ? i : j), n);
9 }

```

## 10.8 AC Automaton

285968

```

1 // Author: std_abs (slightly modified by Gino)
2 // Recap: fail[s] = s 在 trie 上 (也就是某個 pattern 的前綴
  的最長 proper 後綴
3 // N := sum(length of all patterns) + 1
4 // 1. ac.insert(pat) -> end-node id recorded in ac.pid
5 // 2. ac.build_fail()
6 // 3. run text: ptr = to[ptr][c-'a'] each char;
  accumulate on your own arrays
7 // - pull: rep(i ac.order_cnt-1 ~ 0) val[fail[x]] += val[x]
8 // - push: rep(i 0 ~ ac.order_cnt-1) val[x] += val[fail[x]]
9 // Tested @yosupo, N <= 1e6, 323ms on building AC
10 const int N = 1e6 + 5, C = 26;
11 struct AC {
12   int ch[N][C], to[N][C], fail[N], _id, order[N], order_cnt;
13   int np; vector<int> pid; // pattern ID on trie
14   // order[r] := which trie node has BFS order r
15   // fail link tree: (parent) fail[i] <-- i (child)
16   void reset() { _id = np = order_cnt = 0, pid.clear(),
  newnode(); }
17   int newnode() {
18     fill_n(ch[_id], C, 0), fill_n(to[_id], C, 0);
19     fail[_id] = 0; return _id++;
20   }
21   AC () { reset(); }
22   void insert(string s) {
23     int ptr = 0; for (char c : s) {
24       if (!ch[ptr][c - 'a']) ch[ptr][c - 'a'] = newnode();
25       ptr = ch[ptr][c - 'a'];
26     } np++; pid.emplace_back(ptr);
27   }
28   void build_fail() {
29     queue<int> q; order_cnt = 0;
30     for (int i = 0; i < C; ++i) if (ch[0][i])
31       q.push(ch[0][i]), to[0][i] = ch[0][i];
32     while (!q.empty()) {
33       int u = q.front(), q.pop(); order[order_cnt++] = u;
34       for (int i = 0; i < C; ++i) {
35         if (!ch[u][i]) to[u][i] = to[fail[u]][i];
36         else { int v = ch[u][i], k = fail[u];
37               while (k && !ch[k][i]) k = fail[k];
38               if (ch[k][i]) k = ch[k][i];
39               fail[v] = k, to[u][i] = v;
40               q.push(v);
41             } } } }
42   } ac;

```

## 10.9 AC Automaton - Dict Links

a62cfa

```

1 // Author: Gino, paste this in `struct AC {...}`
2 // 3 Modifications to the original AC Automaton:
3 // int np; vector<int> pid; bool is_pattern[N];
4 // fail[_id] = 0, is_pattern[_id] = false; return _id++;
5 // np++; pid.emplace_back(ptr); is_pattern[ptr] = true;
6 int dict[N];
7 void build_dict() { // call this after build_fail()!
8   for (int i = 0; i < order_cnt; i++) {
9     int u = order[i], f = fail[u];
10    dict[u] = is_pattern[f] ? f : dict[f];
11  } }

```

## 10.10 AC Automaton - Counting Patterns

58fd32

```

1 // Author: Gino, paste this in `struct AC {...}`
2 ll val[N];
3 void count_patterns(string& text, vector<string>& patterns) {
4   for (auto& p : patterns) insert(p); build_fail();
5   ranges::fill(val, 0LL);
6   int ptr = 0; for (auto& c : text) {
7     ptr = to[ptr][c - 'a']; val[ptr]++;
8   }
9   for (int x : span(order, order_cnt) | views::reverse)
10    val[fail[x]] += val[x];
11   for (auto& i : pid) cout << val[i] << endl;
12 }

```

## 10.11 AC Automaton - Pattern Position

輸出每個 pattern 在 text 中第一次出現的位置，若不存在則輸出 -1。  
走 AC 自動機時 if (val[ptr] == -1) val[ptr] = i; 接著 pull DP  
if (val[x] == -1) continue;  
if (val[fail[x]] == -1) val[fail[x]] = val[x];  
else val[fail[x]] = min(val[fail[x]], val[x]);

## 11 Geometry

### 11.1 Template

acaf6b

```

1 using T = long long; // or double

```

```

2 const double eps = 1e-8;
3 int sgn(T x) { return (x > eps) - (x < -eps); }
4 struct Pt {
5     T x = 0, y = 0;
6     Pt operator+(Pt a) const { return {x + a.x, y + a.y}; }
7     Pt operator-(Pt a) const { return {x - a.x, y - a.y}; }
8     Pt operator*(T k) const { return {x * k, y * k}; }
9     Pt operator/(T k) const { return {x / k, y / k}; }
10    T operator*(Pt a) const { return x * a.x + y * a.y; }
11    T operator^(Pt a) const { return x * a.y - y * a.x; }
12    bool operator<(Pt a) const
13    { return tie(x, y) < tie(a.x, a.y); }
14    bool operator==(Pt a) const
15    { return !sgn(x - a.x) && !sgn(y - a.y); }
16    //friend ostream& operator<<(ostream& o, Pt a)
17    //{{ return o << "(" << a.x << ", " << a.y << ")"; }
18};
19 struct Line { Pt a, b; }; // a -> b
20 T abs2(Pt a) { return a * a; }
21 double abs(Pt a) { return sqrt(abs2(a)); }
22 int ori(Pt o, Pt a, Pt b) // 1: CCW, -1: CW, 0: collinear
23 { return sgn((a - o) ^ (b - o)); }

```

## 11.2 Basic

8507af

```

1 struct Cir { Pt o; double r; };
2 // ----- 極角排序 -----
3 int ud(Pt a) // 0: upper half (incl. +x axis), 1: lower
4 { return !sgn(a.y) ? sgn(a.x) < 0 : a.y < 0; }
5 bool cmp(Pt a, Pt b) // polar angle in [0, 2PI),
6 { return ud(a) == ud(b) ? sgn(a ^ b) > 0 : ud(a) < ud(b); }
7 ranges::sort(p, [&](Pt a, Pt b) { return cmp(a - o, b - o); });
8 // -----
9 bool sameVec(Pt a, Pt b, int d) // d=0: parallel, d=1: same dir
10 { return !sgn(a ^ b) && sgn(a * b) > d * 2 - 2; }
11 bool sameVec(Line a, Line b, int d)
12 { return sameVec(a.b - a.a, b.b - b.a, d); }
13 Pt rot90(Pt a) { return {-a.y, a.x}; } // CCW
14 Pt ref(Pt a) { return ud(a) ? Pt(-a.x, -a.y) : a; }
15 T dbarea(vector<Pt> p) { // doubled signed area, CCW > 0
16     T res = 0; for (int i = 0; i < sz(p); i++)
17         res += p[i] ^ p[(i+1) % sz(p)];
18     return res; }
19 bool onseg(Pt p, Pt a, Pt b) // p on segment ab?
20 { return !ori(p, a, b) && sgn((a - p) * (b - p)) <= 0; }
21 // ---- T = double ----
22 const double PI = acos(-1);
23 double normalize(double x) // to [0, 2pi)
24 { while (x < -eps) x += PI * 2;
25   while (x > PI * 2 + eps) x -= PI * 2;
26   return x; }
27 double theta(Pt a) { return normalize(atan2(a.y, a.x)); }
28 Pt unit(Pt a) { return a / abs(a); }
29 Pt rot(Pt a, double o) { // CCW
30     double c = cos(o), s = sin(o);
31     return Pt(c * a.x - s * a.y, s * a.x + c * a.y); }
32 Pt projVec(Pt a, Pt b, Pt c) // vector ac proj to ab
33 { return (b - a) * ((c - a) * (b - a)) / abs2(b - a); }
34 Pt projPt(Pt a, Pt b, Pt c) // point c proj to ab
35 { return projVec(a, b, c) + a; }

```

## 11.3 Line / Segment Intersections

7e8b65

```

1 // m=0: segment, m=1: ray from l.a to l.b, m=2: line
2 bool banana(Line ls, int ms, Line lt, int mt, int strict) {
3     auto on = [&](Line l, int m, Pt p) {
4         if (ori(l.a, l.b, p) != 0) return false;
5         if (m && abs2(l.a - p) > abs2(l.b - p)) return true;
6         return m == 2 || sgn((p - l.a) * (p - l.b)) <= -strict;
7     };
8     if (sameVec(ls, lt, 0)) {
9         return on(ls, ms, lt.a) || on(ls, ms, lt.b) ||
10            on(lt, mt, ls.a) || on(lt, mt, ls.b);
11     }
12     auto good = [&](Line l, int m, Line o) {
13         if (m && abs2((l.a - o.a) ^ (l.a - o.b)) >
14             abs2((l.b - o.a) ^ (l.b - o.b))) return true;
15         return m == 2 ||
16            ori(l.a, o.a, o.b) * ori(l.b, o.a, o.b) == -1;
17     };
18     if (good(ls, ms, lt) && good(lt, mt, ls)) return 1;
19     if (!strict) {
20         if (mt != 2 && on(ls, ms, lt.a)) return 1;
21         if (mt == 0 && on(ls, ms, lt.b)) return 1;
22         if (ms != 2 && on(lt, mt, ls.a)) return 1;
23         if (ms == 0 && on(lt, mt, ls.b)) return 1;
24     }
25     return 0;
26 }
27 // becareful when two lines are parallel
28 auto bananaPoint(Line a, Line b) {

```

```

29 auto abc = (a.b - a.a) ^ (b.a - a.a);
30 auto abd = (a.b - a.a) ^ (b.b - a.a);
31 return make_pair(b.b * abc - b.a * abd, abc - abd);
32 }

```

## 11.4 Point-Line / Segment Distance

8b23e4

```

1 // T = double
2 double pointLineDist(Pt p, Line L)
3 { return fabs(((L.b - L.a) ^ (p - L.a)) / abs(L.b - L.a)); }
4 double pointSegDist(Pt p, Pt a, Pt b) {
5     if (a == b) return abs(p - a);
6     if (sgn((b - a) * (p - a)) >= 0 &&
7         sgn((a - b) * (p - b)) >= 0)
8         return pointLineDist(p, {a, b});
9     return min(abs(p - a), abs(p - b));
10 }

```

## 11.5 Sort

63db4f

```

1 // Dependency: cmp(), btwAngle()
2 // cmp in Basic: polar angle sort
3 // all points are on line ab. closer to a: front
4 bool cmpLine(Pt s, Pt t, Pt a, Pt b) {
5     Pt v = a - b;
6     if (sgn(v.x)) return sgn(s.x - t.x) == sgn(v.x);
7     return sgn(s.y - t.y) == sgn(v.y);
8 }
9 // intersect points polar angle sort, deno: positive
10 bool cmpFractionPolar(pair<Pt, ll> o,
11 pair<Pt, ll> s, pair<Pt, ll> t) {
12     Pt u = s.first * o.second - o.first * s.second;
13     Pt v = t.first * o.second - o.first * t.second;
14     return cmp(u, v);
15 }
16 struct Seg {
17     Pt a, b; // a.x < b.x
18     bool operator<(const Seg& o) const {
19         if (a == o.a) return ori(o.b, a, b) == 1;
20         if (a.x <= o.a.x) return ori(o.a, a, b) == 1;
21         return ori(a, o.a, o.b) == -1;
22     };
23 }
24 struct PolarSeg {
25     Pt a, b; // ori(Pt(0, 0), a, b) > 0
26     bool operator<(const PolarSeg& o) const {
27         if (a == o.a) return ori(o.b, a, b) == -1;
28         if (btwAngle(Pt(0, 0), a, b, o.a, 0))
29             return ori(o.a, a, b) == -1;
30         return ori(a, o.a, o.b) == 1;
31     };
32 }
33 struct Arc {
34     // contain(a, b): circle a inside circle b
35     Cir c; int id, s; // 0 -> up, 1 -> down
36     bool operator<(const Arc& b) const {
37         if (id == b.id) return s < b.s;
38         if (contain(c, b.c)) return b.s == 1;
39         if (contain(b.c, c)) return s == 0;
40         if (c.o.y == b.c.o.y) return id < b.id;
41         return c.o.y > b.c.o.y;
42     };
43 }

```

## 11.6 Convex Hull

37fe2b

```

1 auto convexHull(vector<Pt> pts) {
2     sort(all(pts)); vector<Pt> ans = {pts[0]};
3     for (int t = 0; t < 2; t++, reverse(all(pts))) {
4         for (int i = 1, m = sz(ans); i < sz(pts); i++) {
5             while (sz(ans) > m &&
6                 ori(ans[sz(ans) - 2], ans.back(), pts[i]) <= 0)
7                 ans.pop_back();
8             ans.pb(pts[i]); } }
9     if (sz(ans) > 1) ans.pop_back(); return ans; }

```

## 11.7 Triangle Centers

9c22fc

```

1 // T = double
2 Pt circcenter(Pt a, Pt b, Pt c) { // r = abs(o - a)
3     b = b - a, c = c - a; double m = 2 * (b^c);
4     Pt o = {(abs2(b) * c.y - abs2(c) * b.y) / m,
5             (abs2(c) * b.x - abs2(b) * c.x) / m};
6     return o + a;
7 }
8 Pt incenter(Pt a, Pt b, Pt c) { // r = 2 * area / s
9     double x = abs(b - c), y = abs(c - a);
10    double z = abs(a - b), s = x + y + z;
11    return (a*x + b*y + c*z) / s;
12 }
13 Pt centroid(Pt a, Pt b, Pt c) { return (a + b + c) / 3; }
14 Pt orthocenter(Pt a, Pt b, Pt c) {
15     return centroid(a, b, c) * 3 - circcenter(a, b, c) * 2;
16 }

```

## 11.8 Point Inside

109f6b

```

1 // get edge index: check (0, a), (0, b) first
2 // then after binary search, check (a, b)
3 bool inConvex(vector<Pt> C, Pt p, bool strict = true) {

```

```

4 // only works when no three points are collinear
5 int a = 1, b = sz(C) - 1, r = !strict;
6 if (sz(C) == 0) return false;
7 if (sz(C) < 3) return r && onseg(p, C[0], C.back());
8 if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
9 if (ori(C[0], C[a], p) >= r ||
10     ori(C[0], C[b], p) <= -r) return false;
11 while (abs(a - b) > 1) {
12     int c = (a + b) / 2;
13     (ori(C[0], C[c], p) > 0 ? b : a) = c;
14 }
15 return ori(C[a], C[b], p) < r;
16 }
17 // need onseg(), -1: out, 0: edge, 1: in
18 int inPoly(vector<Pt> poly, Pt o, int strict) {
19     int cnt = 0;
20     for (int i = 0; i < sz(poly); i++) {
21         Pt a = poly[i], b = poly[(i + 1) % sz(poly)];
22         if (onseg(o, a, b)) return !strict;
23         cnt ^= ((o.y < a.y) - (o.y < b.y)) * ori(o, a, b) > 0;
24     }
25     return cnt ? 1 : -1;
26 }
27 // relation of q with circumcircle; T = long long
28 bool inCC(array<Pt, 3> p, Pt q) {
29     __int128 det = 0;
30     for (int i = 0; i < 3; i++)
31         det += __int128(abs2(p[i]) - abs2(q)) *
32             ((p[(i + 1) % 3] - q) ^ (p[(i + 2) % 3] - q));
33     return det > 0; // in: >0, on: =0, out: <0
34 }

```

### 11.9 Point In Triangle

144fc0

```

1 // number of p with a < p < b and ori(p, a, b) < 0
2 int under(Pt a, Pt b) {}
3 // number of points with a < p < b and ori(p, a, b) = 0
4 int edge(Pt a, Pt b) {}
5 // check if this number is calculated
6 bool check(Pt p) {}
7 // number of points that strictly inside the triangle
8 int inTri(array<Pt, 3> arr) {
9     sort(all(arr)); auto [a, b, c] = arr; intx = ori(b, a, c);
10    if (x == 0) return 0;
11    if (x == 1) return under(a, b) + under(b, c) -
12        under(a, c) - edge(a, c);
13    return under(a, c) - under(a, b) -
14        under(b, c) - edge(a, b) - edge(b, c) - check(b);
15 }

```

### 11.10 Vector In Polygon

1b2dce

```

1 // ori(a, b, c) >= 0, valid: "strict" angle from a-b to a-c
2 bool btwAngle(Pt a, Pt b, Pt c, Pt p, int strict) {
3     return ori(a, b, p) >= strict &&
4         ori(a, p, c) >= strict;
5 }
6 // whether vector{cur, p} is in order prv, cur, nxt
7 bool inside(Pt prv, Pt cur, Pt nxt, Pt p, int strict) {
8     if (ori(cur, nxt, prv) >= 0)
9         return btwAngle(cur, nxt, prv, p, strict);
10    return !btwAngle(cur, prv, nxt, p, !strict);
11 }

```

### 11.11 Polygon Cut

c08d26

```

1 // Dependency: bananaPoint(); T = double
2 // Cut a convex polygon with line (s -> e), keep right side
3 vector<Pt> polyCut(vector<Pt> poly, Pt s, Pt e) {
4     vector<Pt> res; for (int i = 0; i < sz(poly); i++) {
5         Pt cur = poly[i], prv = i ? poly[i - 1] : poly.back();
6         bool side = ori(s, e, cur) < 0;
7         if (side != (ori(s, e, prv) < 0)) {
8             auto [p, d] = bananaPoint({s, e}, {cur, prv});
9             res.pb(p / d);
10        } if (side) res.pb(cur);
11    } return res;
12 }

```

### 11.12 Half-Plane Intersection

c71070

```

1 // Dependency: sameVec(), cmp(); T = long long
2 /* Having solution, check size > 2 */
3 /* --^-- Line.a --^-- Line.b --^-- */
4 auto halfPlaneIntersection(vector<Line> arr) {
5     auto areaPair = [&](Line a, Line b) {
6         return make_pair((a.b - a.a) ^ (b.a - a.a),
7             (a.b - a.a) ^ (b.b - a.a));
8     };
9     auto isin = [&](Line l0, Line l1, Line l2) {
10        // Check inter(l1, l2) strictly in l0
11        auto [a02X, a02Y] = areaPair(l0, l2);
12        auto [a12X, a12Y] = areaPair(l1, l2);
13        if (a12X - a12Y < 0) a12X *= -1, a12Y *= -1;

```

```

14     return (__int128)a02Y * a12X -
15         (__int128)a02X * a12Y > 0; // C^4
16 };
17 sort(all(arr), [&](Line a, Line b) {
18     if (sameVec(a, b, 1))
19         return ori(a.a, a.b, b.b) < 0;
20     return cmp(a.b - a.a, b.b - b.a);
21 });
22 deque<Line> dq(1, arr[0]);
23 auto popBack = [&](int t, Line p) {
24     while (sz(dq) >= t &&
25         !isin(p, dq[sz(dq) - 2], dq.back()))
26         dq.pop_back();
27 };
28 auto popFront = [&](int t, Line p) {
29     while (sz(dq) >= t && !isin(p, dq[0], dq[1]))
30         dq.pop_front();
31 };
32 for (auto p : arr) if (!sameVec(dq.back(), p, 1))
33     popBack(2, p), popFront(2, p), dq.pb(p);
34 popBack(3, dq[0]), popFront(3, dq.back());
35 return vector<Line>(all(dq));
36 }

```

### 11.13 Cyclic Ternary Search

e3165d

```

1 /* bool pred(int a, int b);
2 f(0) ~ f(n - 1) is a cyclic-shift U-function
3 return idx s.t. pred(x, idx) is false forall x*/
4 int cycTsearch(int n, auto pred) {
5     if (n == 1) return 0;
6     int l = 0, r = n; bool rv = pred(1, 0);
7     while (r - l > 1) {
8         int m = (l + r) / 2;
9         if (pred(0, m) ? rv : pred(m, (m + 1) % n)) r = m;
10        else l = m;
11    } return pred(l, r % n) ? l : r % n;
12 }

```

### 11.14 Circle Intersections

734f65

```

1 // Dependency: cycTsearch(); T = double for circles
2 vector<Pt> circleLineIntersect(Cir c, Line l) {
3     Pt p = l.a + (l.b - l.a) *
4         ((c.o - l.a) * (l.b - l.a)) / abs2(l.b - l.a);
5     double s = (l.b - l.a) ^ (c.o - l.a);
6     double h2 = c.r * c.r - s * s / abs2(l.b - l.a);
7     if (sgn(h2) == -1) return {};
8     if (sgn(h2) == 0) return {p};
9     Pt h = (l.b - l.a) / abs(l.b - l.a) * sqrt(h2);
10    return {p - h, p + h};
11 }
12 // covered area of c1: arc from res[0] to res[1], CCW
13 vector<Pt> circlesIntersect(Cir c1, Cir c2) {
14     double d2 = abs2(c1.o - c2.o), d = sqrt(d2);
15     if (d < max(c1.r, c2.r) - min(c1.r, c2.r) ||
16         d > c1.r + c2.r) return {};
17     Pt u = (c1.o + c2.o) / 2 +
18         (c1.o - c2.o) *
19         ((c2.r * c2.r - c1.r * c1.r) / (2 * d2));
20     double A = sqrt((c1.r + c2.r + d) *
21         (c1.r - c2.r + d) * (c1.r + c2.r - d) *
22         (-c1.r + c2.r + d));
23     Pt v = rot90(c2.o - c1.o) * A / (2 * d2);
24     if (sgn(v.x) == 0 && sgn(v.y) == 0) return {u};
25     return {u - v, u + v};
26 }
27 vector<pii> convexLineIntersect(vector<Pt>& C, Line la) {
28     auto dis = [&](int p) {
29         return (la.b - la.a) ^ (C[p] - la.a);
30     };
31     auto gao = [&](int s) {
32         return cycTsearch(sz(C), [&](int i, int j) {
33             return sgn(dis(i) - dis(j)) == s;
34         });
35     };
36     int x = gao(1), y = gao(-1), n = sz(C);
37     if (sgn(dis(x)) < 0 || sgn(dis(y)) > 0) return {};
38     if (sgn(dis(x)) == 0 || sgn(dis(y)) == 0) {
39         int v = ((sgn(dis(x)) == 0 ? x : y) + n - 1) % n;
40         vector<pii> vec;
41         for (int i = 0; i < 3; i++, v = (v + 1) % n)
42             if (sgn(dis(v)) == 0) vec.emplace_back(v, v);
43         return vec;
44     }
45     auto get = [&](int l, int r, int s) {
46         while ((l + 1) % n != r) {
47             int m = ((l + r + (l < r ? 0 : n)) / 2) % n;
48             if (sgn(dis(m)) == s ? l : r) = m;
49         }
50         if (sgn(dis(r)) == 0) return pii(r, r);
51         return pii(l, r);

```

```

52 };
53 return {get(x, y, 1), get(y, x, -1)};
54 }

```

### 11.15 Tangents

8def22

```

1// Dependency: cycTsearch(); T = double for circle tangents
2auto circlePointTangent(Cir c, Pt p) {
3    vector<Line> res;
4    double dSq = abs2(p - c.o);
5    if (sgn(dSq - c.r * c.r) == 0) {
6        res.push_back({p, p + rot90(p - c.o)});
7    } else if (dSq > c.r * c.r) {
8        double s = dSq - c.r * c.r;
9        Pt v = p + (c.o - p) * s / dSq;
10       Pt u = rot90(c.o - p) * sqrt(s) * c.r / dSq;
11       res.push_back({p, v + u});
12       res.push_back({p, v - u});
13    }
14    return res;
15}
16auto circlesTangent(Cir c1, Cir c2, int sign1) {
17    // sign1 = 1 for outer tang, -1 for inter tang
18    vector<Line> res;
19    double dSq = abs2(c1.o - c2.o);
20    if (sgn(dSq) == 0) return res;
21    double d = sqrt(dSq);
22    Pt v = (c2.o - c1.o) / d;
23    double c = (c1.r - sign1 * c2.r) / d;
24    if (c * c > 1) return res;
25    double h = sqrt(max(0.0, 1.0 - c * c));
26    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
27        Pt n = Pt(v.x * c - sign2 * h * v.y,
28                v.y * c + sign2 * h * v.x);
29        Pt p1 = c1.o + n * c1.r;
30        Pt p2 = c2.o + n * (c2.r * sign1);
31        if (sgn(p1.x - p2.x) == 0 &&
32            sgn(p1.y - p2.y) == 0)
33            p2 = p1 + rot90(c2.o - c1.o);
34        res.push_back({p1, p2});
35    }
36    return res;
37}
38/* The point should be strictly out of hull
39   return arbitrary point on the tangent line */
40pii pointConvexTangent(vector<Pt>& C, Pt p) {
41    auto gao = [&](int s) {
42        return cycTsearch(sz(C), [&](int x, int y) {
43            return ori(p, C[x], C[y]) == s;
44        });
45    };
46    return pii(gao(1), gao(-1));
47} // return (a, b), ori(p, C[a], C[b]) >= 0

```

### 11.16 Bisector

5348b9

```

1Pt bisector(Pt a, Pt b, Pt c) { Pt s = b-a, t = c-a;
2    t = t * abs(s) / abs(t); return s + t; }

```

### 11.17 Minimum Enclosing Circle

04b498

```

1// Dependency: circenter(); T = double
2Cir minEnclosingCircle(vector<Pt> p) {
3    auto seed = chrono::steady_clock::now();
4    time_since_epoch().count();
5    shuffle(all(p), mt19937(seed));
6    double r = 0.0; Pt cent = p[0];
7    for (int i = 1; i < sz(p); i++) {
8        if (abs2(cent - p[i]) <= r) continue;
9        cent = p[i]; r = 0.0;
10       for (int j = 0; j < i; j++) {
11           if (abs2(cent - p[j]) <= r) continue;
12           cent = (p[i] + p[j]) / 2; r = abs2(p[j] - cent);
13           for (int k = 0; k < j; k++) {
14               if (abs2(cent - p[k]) <= r) continue;
15               cent = circenter(p[i], p[j], p[k]);
16               r = abs2(p[k] - cent);
17           }
18       }
19       return {cent, sqrt(r)};
20 }

```

### 11.18 Minkowski Sum

fdd915

```

1// Dependency: convexHull()
2auto minkowski(vector<Pt> a, vector<Pt> b) {
3    a = convexHull(a), b = convexHull(b);
4    int n = sz(a), m = sz(b);
5    vector<Pt> c = {a[0] + b[0]}, s1, s2;
6    for (int i = 0; i < n; i++) s1.pb(a[(i+1) % n] - a[i]);
7    for (int i = 0; i < m; i++) s2.pb(b[(i+1) % m] - b[i]);
8    for (int p1 = 0, p2 = 0; p1 < n || p2 < m;)
9        if (p2 == m ||
10            (p1 < n && sgn(s1[p1] ^ s2[p2]) >= 0))
11            c.pb(c.back() + s1[p1++]);

```

```

12 else
13     c.pb(c.back() + s2[p2++]);
14 return convexHull(c);
15 }

```

### 11.19 Convex Hull Distance

f7df29

```

1// Dependency: minkowski(), inConvex(), pointSegDist()
2// T = double
3double convexHullDist(vector<Pt> A, vector<Pt> B) {
4    Pt O(0, 0); for (auto& p : B) p = 0 - p;
5    auto C = minkowski(A, B);
6    if (inConvex(C, O)) return 0;
7    double ans = pointSegDist(O, C.back(), C[0]);
8    for (int i = 0; i + 1 < sz(C); i++)
9        ans = min(ans, pointSegDist(O, C[i], C[i + 1]));
10   return ans; }

```

### 11.20 Closest Point Pair

ad1bba

```

1// returns {min squared dist, {i, j}}, n >= 2, |coord| < 1e9
2pair<T, pair<int, int>> closestPair(vector<Pt> p) {
3    vector<int> id(sz(p)); iota(all(id), 0);
4    sort(all(id),
5         [&](int a, int b) { return p[a].y < p[b].y; });
6    set<pair<Pt, int>> s; // (point, id)
7    pair<T, pair<int, int>> best = {(T)9e18, {-1, -1}};
8    int l = 0; for (int i : id) {
9        if (!best.first) break;
10       T d = sqrtl(best.first) + is_integral_v<T>;
11       while (p[id[l]].y <= p[i].y - d)
12           s.erase({p[id[l]], id[l]}), l++;
13       auto lo = s.lower_bound({p[i] - Pt{d, 0}, 0});
14       auto hi = s.upper_bound({p[i] + Pt{d, 0}, sz(p)});
15       for (; lo != hi; lo++) {
16           auto [q, j] = *lo;
17           best = min(best,
18                     {abs2(q - p[i]), {min(i, j), max(i, j)}});
19       } s.insert({p[i], i});
20   } return best;
21 }

```

### 11.21 Polygon-Circle Intersection

18454b

```

1// T = double
2double areaSector(Pt pa, Pt pb, double r) {
3    if (abs(pa) < abs(pb)) swap(pa, pb);
4    if (abs(pb) < eps) return 0;
5    double S, h, t;
6    double a = abs(pb), b = abs(pa), c = abs(pb - pa);
7    double cosB = pb * (pb - pa) / a / c;
8    double B = acos(cosB);
9    double cosC = pa * pb / a / b, C = acos(cosC);
10   if (a > r) {
11       S = C / 2 * r * r;
12       h = a * b * sin(C) / c;
13       if (h < r && B < PI / 2)
14           S -= acos(h / r) * r * r - h * sqrt(r * r - h * h);
15   } else if (b > r) {
16       t = PI - B - asin(sin(B) / r * a);
17       S = 0.5 * a * r * sin(t) + (C - t) / 2 * r * r;
18   } else {
19       S = 0.5 * sin(C) * a * b;
20   }
21   return S;
22 }
23double areaPolyCircle(vector<Pt> poly, Pt O, double r) {
24   double S = 0;
25   int n = sz(poly);
26   for (int i = 0; i < n; i++)
27       S += areaSector(poly[i] - O,
28                       poly[(i + 1) % n] - O, r) *
29       ori(O, poly[i], poly[(i + 1) % n]);
30   return fabs(S);
31 }

```

### 11.22 Union of Polygons

9d7265

```

1//Author:Gino, simple polygons any orientation, O((sum n)^2)
2// T = ll: exact (|coord| <= 1e9); T = double: also works
3double polysUnionArea(vector<vector<Pt>> poly) {
4    struct Fr { T p, q; }; // p / q, q > 0
5    auto lt = [](Fr a, Fr b) {
6        //return ( (__int128)a.p * b.q < ( (__int128)b.p * a.q; // if T
6        // == long long
7        return a.p * b.q < b.p * a.q;
8    };
9    auto val = [&](Fr a) { // clamp to [0, 1], to double
10        if (lt(a, {0, 1})) return 0.;
11        if (lt({1, 1}, a)) return 1.;
12        return (double)a.p / a.q;
13    };
14    auto get = [&](int i, int e) {
15        return poly[i][(e + 1) % sz(poly[i])];
16    };

```

```

17 for (auto& p : poly)
18     if (dbarea(p) < 0) reverse(all(p));
19 double sum = 0;
20 for (int i = 0; i < sz(poly); i++)
21     for (int e = 0; e < sz(poly[i]); e++) {
22         Pt s = poly[i][e], t = get(i, e);
23         vector<pair<Fr, int>> c = {{0, 1}, {1, 1}, {0}};
24         for (int j = 0; j < sz(poly); j++) {
25             if (i == j) continue;
26             for (int f = 0; f < sz(poly[j]); f++) {
27                 Pt a = poly[j][f], b = get(j, f);
28                 int ta = ori(s, t, a), tb = ori(s, t, b);
29                 if (!ta && !tb) {
30                     if (j < i && sgn((b-a) * (t-s)) > 0) {
31                         T d = abs2(t-s);
32                         c.pb({Fr{(a-s) * (t-s), d}, 1});
33                         c.pb({Fr{(b-s) * (t-s), d}, -1});
34                     }
35                     else if ((ta >= 0) != (tb >= 0)) {
36                         T x = (b-a) ^ (s-a), y = (b-a) ^ (t-a);
37                         if (x < y) x = -x, y = -y;
38                         c.pb({Fr{x, x-y}, ta >= 0 ? 1 : -1});
39                     }
40                 }
41             }
42         }
43         sort(all(c), [&](auto& u, auto& v)
44             { return lt(u.first, v.first); });
45         double z = val(c[0].first), r = 0;
46         for (int k = 1, d = c[0].second; k < sz(c); k++) {
47             double w = val(c[k].first);
48             if (!d) r += w - z;
49             d += c[k].second, z = w;
50         }
51         sum += (s^t) * r;
52     }
53 return sum / 2;

```

### 11.23 Rotating Sweep Line

769b9b

```

1 // Dependency: ref()
2 struct Event {
3     Pt d; int u, v;
4     bool operator<(const Event& o) const {
5         return sgn(d ^ o.d) > 0;
6     };
7 void rotatingSweepLine(vector<Pt> pts) {
8     int n = sz(pts);
9     vector<int> ord(n), pos(n); vector<Event> e;
10    for (int i = 0; i < n; i++)
11        for (int j = i + 1; j < n; j++)
12            e.push_back({ref(pts[i] - pts[j]), i, j});
13    sort(all(e)); iota(ord, 0);
14    sort(all(ord), [&](int i, int j) {
15        return sgn(pts[i].y - pts[j].y) == 0 ?
16            pts[i].x < pts[j].x : pts[i].y < pts[j].y;
17    });
18    for (int i = 0; i < n; i++) pos[ord[i]] = i;
19    auto makeReverse = [](auto v) {
20        sort(all(v));
21        v.resize(unique(all(v)) - v.begin());
22        vector<pii> segs;
23        for (int i = 0, j = 0; i < sz(v); i = j) {
24            for (; j < sz(v) && v[j] - v[i] <= j - i; j++);
25            segs.emplace_back(v[i], v[j - 1] + 1 + 1);
26        }
27        return segs;
28    };
29    for (int i = 0, j = 0; i < sz(e); i = j) {
30        vector<int> tmp;
31        for (; j < sz(e) && !(e[i] < e[j]); j++)
32            tmp.pb(min(pos[e[j].u], pos[e[j].v]));
33        for (auto [l, r] : makeReverse(tmp)) {
34            reverse(ord.begin() + l, ord.begin() + r);
35            for (int t = l; t < r; t++) pos[ord[t]] = t;
36            // update value here
37        }
38    }

```

### 11.24 Pick's Theorem

Consider a polygon which vertices are all lattice points.  
 Let  $i$  = number of points inside the polygon.  
 Let  $b$  = number of points on the boundary of the polygon.  
 Then we have the following formula:

$$\text{Area} = i + b/2 - 1$$

## 12 Number Theory

### 12.1 Mod Sum

60beb8

```

1 /**
2 * Author: Simon Lindholm

```

```

3 * Date: 2015-06-23
4 * License: CC0
5 * Source: own work
6 * Description: Sums of mod'ed arithmetic progressions.
7 *
8 * \texttt{modsum(to, c, k, m)} = \sum_{i=0}^{(k+m-1)/(k+c)} \texttt{modsum}(to + i * (k+c), c, k, m)$.
9 * \texttt{divsum} is similar but for floored division.
10 * Time:  $\log(m)$ , with a large constant.
11 * Status: Tested for all  $|k|, |c|, to, m \leq 50$ , and on kattis:aladin
12 */
13 typedef unsigned long long ull;
14 ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
15 /// ^ written in a weird way to deal with overflows correctly
16 ull divsum(ull to, ull c, ull k, ull m) {
17     ull res = k / m * sumsq(to) + c / m * to;
18     k %= m; c %= m;
19     if (!k) return res;
20     ull to2 = (to * k + c) / m;
21     return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
22 }
23 ll modsum(ull to, ll c, ll k, ll m) {
24     c = ((c % m) + m) % m;
25     k = ((k % m) + m) % m;
26     return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
27 }

```

### 12.2 Extended Lucas Theorem

3bed10

```

1 /**
2 * Author: koukanni
3 * Date: 2025
4 * License: CC0
5 * Source:
6 * Description: Computes  $C(n, m) \bmod P$  where  $P$  is not necessarily prime
7 * Functions
8 * exgcd(a, b, x, y): Extended Euclidean algorithm
9 * inv(a, p): Modular inverse of a modulo p
10 * qpow(a, b, p): Fast modular exponentiation
11 * CRT(n, a, m): Chinese Remainder Theorem
12 * calc(n, x, P): Helper function for computing factorial with prime power
13 * multilucas(n, m, x, P): Lucas theorem for prime power modulus
14 * exlucas(n, m, P): Main function - Extended Lucas theorem
15 * usage: exlucas(n, m, P) returns  $C(n, m) \bmod P$ 
16 * Time:  $O(P \log(P))$ 
17 * Status: tested
18 */
19 typedef long long ll;
20 void exgcd(ll a, ll b, ll& x, ll& y) {
21     if (!b)
22         x = 1, y = 0;
23     else
24         exgcd(b, a % b, y, x), y -= a / b * x;
25 }
26 ll inv(ll a, ll p) {
27     ll x, y;
28     exgcd(a, p, x, y);
29     return (x + p) % p;
30 }
31 ll qpow(ll a, ll b, ll p) {
32     ll r = 1;
33     for (; b >= 1; a = a * a % p)
34         if (b & 1) r = r * a % p;
35     return r;
36 }
37 ll CRT(int n, ll* a, ll* m) {
38     ll M = 1, p = 0;
39     for (int i = 1; i <= n; i++) M = M * m[i];
40     for (int i = 1; i <= n; i++) {
41         ll w = M / m[i], x, y;
42         exgcd(w, m[i], x, y);
43         p = (p + a[i] * w * x % M) % M;
44     }
45     return (p % M + M) % M;
46 }
47 ll calc(ll n, ll x, ll P) {
48     if (!n) return 1;
49     ll s = 1;
50     for (ll i = 1; i <= P; i++)
51         if (i % x) s = s * i % P;
52     s = qpow(s, n / P, P);
53     for (ll i = n / P * P + 1; i <= n; i++)
54         if (i % x) s = i % P * s % P;
55     return s * calc(n / x, x, P) % P;
56 }

```

```

57 ll multilucas(ll n, ll m, ll x, ll P) {
58     if (n < m) return 0;
59     int cnt = 0;
60     for (ll i = n; i; i /= x) cnt += i / x;
61     for (ll i = m; i; i /= x) cnt -= i / x;
62     for (ll i = n - m; i; i /= x) cnt -= i / x;
63     return qpow(x, cnt, P) % P * calc(n, x, P) % P *
        inv(calc(m, x, P), P) % P *
        inv(calc(n - m, x, P), P) % P;
64 }
65
66 ll exlucas(ll n, ll m, ll P) {
67     int cnt = 0;
68     ll p[20], a[20];
69     for (ll i = 2; i * i <= P; i++) {
70         if (P % i == 0) {
71             p[++cnt] = i;
72             while (P % i == 0) p[cnt] = p[cnt] * i, P /= i;
73             a[cnt] = multilucas(n, m, i, p[cnt]);
74         }
75     }
76     if (P > 1) p[++cnt] = P, a[cnt] = multilucas(n, m, P, P);
77     return CRT(cnt, a, p);
78 }

```

## 12.3 Prime Sieve and Defactor

e2350f

```

1 // Author: Gino
2 const int maxc = 1e6 + 1;
3 vector<int> lpf;
4 vector<int> prime;
5
6 void seive() {
7     prime.clear();
8     lpf.resize(maxc, 1);
9     for (int i = 2; i < maxc; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i;
12             prime.emplace_back(i);
13         }
14         for (auto& j : prime) {
15             if (i * j >= maxc) break;
16             lpf[i * j] = j;
17             if (j == lpf[i]) break;
18         }
19     }
20
21 vector<pii> fac;
22 void defactor(int u) {
23     fac.clear();
24     while (u > 1) {
25         int d = lpf[u];
26         fac.emplace_back(make_pair(d, 0));
27         while (u % d == 0) {
28             u /= d;
29             fac.back().second++;
30         }
31     }
32 }

```

## 12.4 Harmonic Series

7c56cc

```

1 // Author: Gino
2 // O(n log n)
3 for (int i = 1; i <= n; i++) {
4     for (int j = i; j <= n; j += i) {
5         // O(1) code
6     }
7 }
8 // PIE
9 // given array a[0], a[1], ..., a[n - 1]
10 // calculate dp[x] = number of pairs (a[i], a[j]) such that
11 // gcd(a[i], a[j]) = x // (i < j)
12 //
13 // idea: let mc(x) = # of y s.t. x|y
14 // f(x) = # of pairs s.t. gcd(a[i], a[j]) >= x
15 // f(x) = C(mc(x), 2)
16 // dp[x] = f(x) - sum(dp[y], x < y and x|y)
17 const int maxc = 1e6;
18 vector<int> cnt(maxc + 1, 0), dp(maxc + 1, 0);
19 for (int i = 0; i < n; i++)
20     cnt[a[i]]++;
21 for (int x = maxc; x >= 1; x--) {
22     ll cnt_mul = 0; // number of multiples of x
23     for (int y = x; y <= maxc; y += x)
24         cnt_mul += cnt[y];
25     dp[x] = cnt_mul * (cnt_mul - 1) / 2; // number of pairs
26     // that are divisible by x
27     for (int y = x + x; y <= maxc; y += x)
28         dp[x] -= dp[y]; // PIE: subtract all dp[y] for y >
29 // x and x|y
30 }

```

## 12.5 Count Number of Divisors

0a5616

```

1 // Author: Gino
2 // Function: Count the number of divisors for all x <= 10^6
3 // using harmonic series
4 const int maxc = 1e6;

```

```

4 vector<int> facs;
5 void find_all_divisors() {
6     facs.clear(); facs.resize(maxc + 1, 0);
7     for (int x = 1; x <= maxc; x++) {
8         for (int y = x; y <= maxc; y += x) {
9             facs[y]++;
10         }
11     }
12 }

```

## 12.6 數論分塊

cd4e6b

```

1 // Author: Gino
2 /*
3 n = 17
4 i: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
5 n/i: 17 8 5 4 3 2 2 2 1 1 1 1 1 1 1 1 1
6
7 L(2) R(2)
8 L(x) := left bound for n/i = x
9 R(x) := right bound for n/i = x
10 ===== FORMULA =====
11 >>> R = n / (n/L) <<<
12 =====
13 Example: L(2) = 6
14 R(2) = 17 / (17 / 6)
15         = 17 / 2
16         = 8
17 */
18 // ===== CODE =====
19 for (ll l = 1, r = 1, q = n; l <= n; l = r + 1) {
20     q = n/l;
21     r = n/q;
22     // Process your code here
23 }
24 // q, l, r: 17 1 1
25 // q, l, r: 8 2 2
26 // q, l, r: 5 3 3
27 // q, l, r: 4 4 4
28 // q, l, r: 3 5 5
29 // q, l, r: 2 6 8
30 // q, l, r: 1 9 17

```

## 12.7 Pollard's rho

```

1 // Author: Unknown
2 // Function: Find a non-trivial factor of a big number in
3 // O(n^(1/4) log^2(n))
4 ll find_factor(ll number) {
5     __int128 x = 2;
6     for (__int128 cycle = 1; ; cycle++) {
7         __int128 y = x;
8         for (int i = 0; i < (1 << cycle); i++) {
9             x = (x * x + 1) % number;
10            __int128 factor = __gcd(x - y, number);
11            if (factor > 1)
12                return factor;
13        }
14    }
15 }

```

```

1 # Author: Unknown
2 # Function: Find a non-trivial factor of a big number in
3 // O(n^(1/4) log^2(n))
4 from itertools import count
5 from math import gcd
6 from sys import stdin
7 for s in stdin:
8     number, x = int(s), 2
9     brk = False
10    for cycle in count(1):
11        y = x
12        if brk:
13            break
14        for i in range(1 << cycle):
15            x = (x * x + 1) % number
16            factor = gcd(x - y, number)
17            if factor > 1:
18                print(factor)
19                brk = True
20                break

```

## 12.8 Miller Rabin

686187

```

1 // Author: Unknown, Modified by Gino
2 // Function: Check if a number is a prime in O(100 *
3 // log^2(n))
4 // miller_rabin(): return 1 if prime, 0 otherwise
5 inline ll mul(ll x, ll y, ll mod) {
6     return (__int128)(x) * y % mod;
7 }
8 ll mypow(ll a, ll b, ll mod) {
9     ll r = 1;

```

```

9   while (b > 0) {
10       if (b & 1) r = mul(r, a, mod);
11       a = mul(a, a, mod);
12       b >>= 1;
13   }
14   return r;
15 }
16 bool witness(ll a, ll n, ll u, int t){
17     ll x = mypow(a, u, n);
18     for(int i = 0; i < t; i++) {
19         ll nx = mul(x, x, n);
20         if (nx == 1 && x != 1 && x != n-1) return true;
21         x = nx;
22     }
23     return x != 1;
24 }
25 bool miller_rabin(ll n) {
26     // if n >= 3,474,749,660,383
27     // change {2, 3, 5, 7, 11, 13} to
28     // {2, 325, 9375, 28178, 450775, 9780504, 1795265022}
29     if (n < 2) return false;
30     if (!(n & 1)) return n == 2;
31     ll u = n - 1; int t = 0;
32     while (!(u & 1)) u >>= 1, t++;
33     for (ll a : {2, 3, 5, 7, 11, 13}) {
34         if (a % n == 0) continue;
35         if (witness(a, n, u, t)) return false;
36     }
37     return true;
38 }
39 // bases that make sure no pseudoprimes flee from test
40 // if WA, replace randll(n - 1) with these bases:
41 // n < 4,759,123,141      3 : 2, 7, 61
42 // n < 1,122,004,669,633  4 : 2, 13, 23, 1662803
43 // n < 3,474,749,660,383  6 : pirmses <= 13
44 // n < 2^64              7 :
45 // 2, 325, 9375, 28178, 450775, 9780504, 1795265022

```

## 12.9 Discrete Log

a04e88

```

1 // Author: ckiseki
2 // find x^? = y (mod M)
3 // O(sqrt(M))
4 template<typename Int>
5 Int BSGS(Int x, Int y, Int M) {
6     // x^? \equiv y (mod M)
7     Int t = 1, c = 0, g = 1;
8     for (Int M_ = M; M_ > 0; M_ >>= 1) g = g * x % M;
9     for (g = gcd(g, M); t % g != 0; ++c) {
10         if (t == y) return c;
11         t = t * x % M;
12     }
13     if (y % g != 0) return -1;
14     t /= g, y /= g, M /= g;
15     Int h = 0, gs = 1;
16     for (; h * h < M; ++h) gs = gs * x % M;
17     unordered_map<Int, Int> bs;
18     for (Int s = 0; s < h; bs[y] = ++s) y = y * x % M;
19     for (Int s = 0; s < M; s += h) {
20         t = t * gs % M;
21         if (bs.count(t)) return c + s + h - bs[t];
22     }
23     return -1;
24 }

```

## 12.10 Discrete Sqrt

3f16e1

```

1 // Author: ckiseki
2 // find solution for x^2 = n (mod P)
3 // O(log^2 P)
4 int get_root(int n, int P) { // ensure 0 <= n < P
5     if (P == 2 || n == 0) return n;
6     auto check = [&](ll x) {
7         return modpow(x, (P - 1) / 2, P);
8     };
9     if (check(n) != 1) return -1;
10    mt19937 rnd(7122); ll z = 1, w;
11    while (check(w = (z * z - n + P) % P) != P - 1)
12        z = rnd() % P;
13    const auto M = [P, w](auto &u, auto &v) {
14        auto [a, b] = u; auto [c, d] = v;
15        return make_pair((a * c + b * d % P * w) % P,
16                          (a * d + b * c) % P);
17    };
18    pair<ll, ll> r(1, 0), e(z, 1);
19    for (int q = (P + 1) / 2; q; q >>= 1, e = M(e, e))
20        if (q & 1) r = M(r, e);
21    return int(r.first); // sqrt(n) mod P where P is prime

```

## 12.11 Fast Power

Note:  $a^n \equiv a^{(n \bmod (p-1))} \pmod{p}$

## 12.12 Extend GCD

50b7fc

```

1 // Author: Gino
2 // [Usage]
3 // bezout(a, b, c):
4 //     find solution to ax + by = c
5 //     return {-LINF, -LINF} if no solution
6 // inv(a, p):
7 //     find modulo inverse of a under p
8 //     return -1 if not exist
9 // CRT(vector<ll>& a, vector<ll>& m)
10 //     find a solution pair (x, mod) satisfies all x = a[i]
11 //         (mod m[i])
12 //     return {-LINF, -LINF} if no solution
13 const ll LINF = 4e18;
14 typedef pair<ll, ll> pll;
15 template<typename T1, typename T2>
16 T1 chmod(T1 a, T2 m) {
17     return (a % m + m) % m;
18 }
19
20 ll GCD;
21 pll extgcd(ll a, ll b) {
22     if (b == 0) {
23         GCD = a;
24         return pll{1, 0};
25     }
26     pll ans = extgcd(b, a % b);
27     return pll{ans.second, ans.first - a/b * ans.second};
28 }
29 pll bezout(ll a, ll b, ll c) {
30     bool negx = (a < 0), negy = (b < 0);
31     pll ans = extgcd(abs(a), abs(b));
32     if (c % GCD != 0) return pll{-LINF, -LINF};
33     return pll{ans.first * c/GCD * (negx ? -1 : 1),
34               ans.second * c/GCD * (negy ? -1 : 1)};
35 }
36 ll inv(ll a, ll p) {
37     if (p == 1) return -1;
38     pll ans = bezout(a % p, -p, 1);
39     if (ans == pll{-LINF, -LINF}) return -1;
40     return chmod(ans.first, p);
41 }
42 pll CRT(vector<ll>& a, vector<ll>& m) {
43     for (int i = 0; i < (int)a.size(); i++)
44         a[i] = chmod(a[i], m[i]);
45
46     ll x = a[0], mod = m[0];
47     for (int i = 1; i < (int)a.size(); i++) {
48         pll sol = bezout(mod, m[i], a[i] - x);
49         if (sol.first == -LINF) return pll{-LINF, -LINF};
50
51         // prevent long long overflow
52         ll p = chmod(sol.first, m[i] / GCD);
53         ll lcm = mod / GCD * m[i];
54         x = chmod((__int128)p * mod + x, lcm);
55         mod = lcm;
56     }
57     return pll{x, mod};
58 }

```

## 12.13 Mu + Phi

ab4a91

```

1 // Author: Gino
2 const int maxn = 1e6 + 5;
3 ll f[maxn];
4 vector<int> lpf, prime;
5 void build() {
6     lpf.clear(); lpf.resize(maxn, 1);
7     prime.clear();
8     f[1] = ...; /* mu[1] = 1, phi[1] = 1 */
9     for (int i = 2; i < maxn; i++) {
10         if (lpf[i] == 1) {
11             lpf[i] = i; prime.emplace_back(i);
12             f[i] = ...; /* mu[i] = 1, phi[i] = i-1 */
13         }
14         for (auto& j : prime) {
15             if (i*j >= maxn) break;
16             lpf[i*j] = j;
17             if (i % j == 0) f[i*j] = ...; /* 0, phi[i]*j */
18             else f[i*j] = ...; /* -mu[i], phi[i]*phi[j] */
19             if (j >= lpf[i]) break;
20         }
21     }
22 }

```

## 12.14 Other Formulas

- Pisano Period: 任何線性遞迴 (比如費氏數列) 模任何一個數字  $M$  都會循環, 找循環節  $\pi(M)$  先質因數分解  $M = \prod p_i^{e_i}$ , 然後  $\pi(M) = \text{lcm}(\pi(p_i^{e_i}))$ ,
- Inversion:  $aa^{-1} \equiv 1 \pmod{m}$ .  $a^{-1}$  exists iff  $\text{gcd}(a, m) = 1$ .

- Linear inversion:  
 $a^{-1} \equiv (m - \lfloor \frac{m}{a} \rfloor) \times (m \bmod a)^{-1} \pmod{m}$
- Fermat's little theorem:  
 $a^p \equiv a \pmod{p}$  if  $p$  is prime.
- Euler function:  
 $\varphi(n) = n \prod_{p|n} \frac{p-1}{p}$
- Euler theorem:  
 $a^{\varphi(n)} \equiv 1 \pmod{n}$  if  $\gcd(a, n) = 1$ . If  $a, n$  are not coprime: 質因數分解  $n = \prod p_i^{e_i}$ , 對每個  $p_i^{e_i}$  分開看他們跟  $a$  是否互質 (互質 : Fermat / 不互質 : 夠大的指數會直接削成 0), 最後用 CRT 合併。
- Extended Euclidean algorithm:  
 $ax + by = \gcd(a, b) = \gcd(b, a \bmod b) = \gcd(b, a - \lfloor \frac{a}{b} \rfloor b) = bx_1 + (a - \lfloor \frac{a}{b} \rfloor b)y_1 = ay_1 + b(x_1 - \lfloor \frac{a}{b} \rfloor y_1)$
- Divisor function:  
 $\sigma_x(n) = \sum_{d|n} d^x \cdot n = \prod_{i=1}^r p_i^{a_i}$   
 $\sigma_x(n) = \prod_{i=1}^r \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$  if  $x \neq 0$ .  $\sigma_0(n) = \prod_{i=1}^r (a_i + 1)$ .
- Chinese remainder theorem (Coprime Moduli):  
 $x \equiv a_i \pmod{m_i}$ .  
 $M = \prod m_i$ .  $M_i = \frac{M}{m_i}$ .  $t_i = M_i^{-1}$ .  
 $x = kM + \sum a_i t_i M_i$ ,  $k \in \mathbb{Z}$ .
- Chinese remainder theorem:  
 $x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2} \Rightarrow x = m_1 p + a_1 = m_2 q + a_2 \Rightarrow m_1 p - m_2 q = a_2 - a_1$   
Solve for  $(p, q)$  using ExtGCD.  
 $x \equiv m_1 p + a_1 \equiv m_2 q + a_2 \pmod{\text{lcm}(m_1, m_2)}$
- Avoiding Overflow:  $ca \bmod cb = c(a \bmod b)$
- Dirichlet Convolution:  $(f * g)(n) = \sum_{d|n} f(n)g(\frac{n}{d})$
- Important Multiplicative Functions + Properties:
  - $\varepsilon(n) = [n = 1]$
  - $1(n) = 1$
  - $id(n) = n$
  - $\mu(n) = 0$  if  $n$  has squared prime factor
  - $\mu(n) = (-1)^k$  if  $n = p_1 p_2 \dots p_k$
  - $\varepsilon = \mu * 1$
  - $\varphi = \mu * id$
  - $[n = 1] = \sum_{d|n} \mu(d)$
  - $[gcd = 1] = \sum_{d|gcd} \mu(d)$
- Möbius inversion:  $f = g * 1 \Leftrightarrow g = f * \mu$

## 12.15 Polynomial

57318b

```

1// Author: Gino
2// Preparation: first set_mod(mod, g), then init_ntt()
3// everytime you change modulus, you have to call
  init_ntt() again
4//
5// [Usage]
6// polynomial: vector<ll> a, b
7// negation: -a
8// add/subtract: a += b, a -= b
9// convolution: a *= b
10// in-place modulo: mod(a, b)
11// in-place inversion under mod x^N: inv(ia, N)
12const int maxk = 20;
13const int maxn = 1<<maxk;
14using u64 = unsigned long long;
15using u128 = __uint128_t;
16int g;
17u64 MOD;
18u64 BARRETT_IM; // [ 2^64 / MOD ]
19inline void set_mod(u64 m, int _g) {
20    g = _g;
21    MOD = m;
22    BARRETT_IM = (u128(1) << 64) / m;
23}
24inline u64 chmod(u128 x) {
25    u64 q = (u64)((x * BARRETT_IM) >> 64);
26    u64 r = (u64)(x - (u128)q * MOD);
27    if (r >= MOD) r -= MOD;
28    return r;
29}
30inline u64 mmul(u64 a, u64 b) {
31    return chmod((u128)a * b);
32}
33ll pw(ll a, ll n) {
34    ll ret = 1;
35    while (n > 0) {
36        if (n & 1) ret = mmul(ret, a);
37        a = mmul(a, a);
38        n >>= 1;
39    }
40    return ret;
41}

```

```

42vector<ll> X, iX;
43vector<int> rev;
44void init_ntt() {
45    X.assign(maxn, 1); // x1 = g^((p-1)/n)
46    iX.assign(maxn, 1);
47    ll u = pw(g, (MOD-1)/maxn);
48    ll iu = pw(u, MOD-2);
49    for (int i = 1; i < maxn; i++) {
50        X[i] = mmul(X[i-1], u);
51        iX[i] = mmul(iX[i-1], iu);
52    }
53    if ((int)rev.size() == maxn) return;
54    rev.assign(maxn, 0);
55    for (int i = 1, hb = -1; i < maxn; i++) {
56        if (!(i & (i-1))) hb++;
57        rev[i] = rev[i ^ (1<<hb)] | (1<<(maxk-hb-1));
58    }
59template<typename T>
60void NTT(vector<T>& a, bool inv=false) {
61    int _n = (int)a.size();
62    int k = __lg(_n) + ((1<<__lg(_n)) != _n);
63    int n = 1<<k;
64    a.resize(n, 0);
65    short shift = maxk-k;
66    for (int i = 0; i < n; i++)
67        if (i > (rev[i]>>shift))
68            swap(a[i], a[rev[i]>>shift]);
69    for (int len = 2, half = 1, div = maxn>>1; len <= n;
70        len<<=1, half<<=1, div>>=1) {
71        for (int i = 0; i < n; i += len) {
72            for (int j = 0; j < half; j++) {
73                T u = a[i+j];
74                T v = mmul(a[i+j+half], (inv ? iX[j*div] :
75                    X[j*div]));
76                a[i+j] = (u+v >= MOD ? u+v-MOD : u+v);
77                a[i+j+half] = (u-v < 0 ? u-v+MOD : u-v);
78            }
79        }
80        if (inv) {
81            T dn = pw(n, MOD-2);
82            for (auto& x : a) {
83                x = mmul(x, dn);
84            }
85        }
86    }
87template<typename T>
88inline void shrink(vector<T>& a) {
89    int cnt = (int)a.size();
90    for (; cnt > 0; cnt--) if (a[cnt-1]) break;
91    a.resize(max(cnt, 1));
92}
93template<typename T>
94vector<T>& operator*=(vector<T>& a, vector<T> b) {
95    int na = (int)a.size();
96    int nb = (int)b.size();
97    a.resize(na + nb - 1, 0);
98    b.resize(na + nb - 1, 0);
99    NTT(a); NTT(b);
100    for (int i = 0; i < (int)a.size(); i++)
101        a[i] = mmul(a[i], b[i]);
102    NTT(a, true);
103    shrink(a);
104    return a;
105}
106inline ll crt(ll a0, ll a1, ll m1, ll m2, ll
107    inv_m1_mod_m2){
108    // x ≡ a0 (mod m1), x ≡ a1 (mod m2)
109    // t = (a1 - a0) * inv(m1) mod m2
110    // x = a0 + t * m1 (mod m1*m2)
111    ll t = chmod(a1 - a0);
112    if (t < 0) t += m2;
113    t = (ll)((__int128)t * inv_m1_mod_m2 % m2);
114    return a0 + (ll)((__int128)t * m1);
115}
116void mul_crt() {
117    // a copy to a1, a2 | b copy to b1, b2
118    ll M1 = 998244353, M2 = 1004535809;
119    g = 3; set_mod(M1); init_ntt(); a1 *= b1;
120    g = 3, set_mod(M2); init_ntt(); a2 *= b2;
121    ll inv_m1_mod_m2 = pw(M1, M2 - 2);
122    for (int i = 2; i <= 2 * k; i++)
123        cout << crt(a1[i], a2[i], M1, M2, inv_m1_mod_m2)
124        << ' ';
125    cout << endl;
126}
127/* P = r*2^k + 1
128P          r      k      g
129998244353 119 23 3
1301004535809 479 21 3
131P          r      k      g
1323          1      1 2
1335          1      2 2

```

```

127 17      1  4  3
128 97      3  5  5
129 193     3  6  5
130 257     1  8  3
131 7681    15  9 17
132 12289   3 12 11
133 40961   5 13  3
134 65537   1 16  3
135 786433  3 18 10
136 5767169 11 19  3
137 7340033  7 20  3
138 23068673 11 21  3
139 104857601 25 22  3
140 167772161 5 25  3
141 469762049 7 26  3
142 1004535809 479 21  3
143 2013265921 15 27 31
144 2281701377 17 27  3
145 3221225473 3 30  5
146 75161927681 35 31  3
147 77309411329 9 33  7
148 206158430209 3 36 22
149 2061584302081 15 37  7
150 2748779069441 5 39  3
151 6597069766657 3 41  5
152 39582418599937 9 42  5
153 79164837199873 9 43  5
154 263882790666241 15 44  7
155 1231453023109121 35 45  3
156 1337006139375617 19 46  3
157 3799912185593857 27 47  5
158 4222124650659841 15 48 19
159 7881299347898369 7 50  6
160 31525197391593473 7 52  3
161 180143985094819841 5 55  6
162 1945555039024054273 27 56  5
163 4179340454199820289 29 57  3
164 9097271247288401921 505 54  6 */

```

## 12.16 Counting Primes

1662d0

```

1 // prime_count - #primes in [1..n] (O(n^{2/3}) time,
2 // O(sqrt(n)) memory)
3 using u64 = unsigned long long;
4 static inline u64 prime_count(u64 n){
5     if(n<=1) return 0;
6     int v = (int)floor(sqrt((long double)n));
7     int s = (v+1)>>1, pc = 0;
8     vector<int> smalls(s), roughs(s), skip(v+1);
9     vector<long long> larges(s);
10    for(int i=0; i<s; ++i){
11        smalls[i]=i;
12        roughs[i]=2*i+1;
13        larges[i]=(long long)((n/roughs[i]-1)>>1);
14    }
15    for(int p=3; p<=v; p+=2) if(!skip[p]){
16        int q = p*p;
17        if(1LL*q*q > (long long)n) break;
18        skip[p]=1;
19        for(int i=q; i<=v; i+=2*p) skip[i]=1;
20        int ns=0;
21        for(int k=0; k<s; ++k){
22            int i = roughs[k];
23            if(skip[i]) continue;
24            u64 d = (u64)i * (u64)p;
25            long long sub = (d <= (u64)v)
26                ? larges[smalls[(int)(d>>1)] - pc]
27                : smalls[(int)((n/d - 1) >> 1)];
28            larges[ns] = larges[k] - sub + pc;
29            roughs[ns++] = i;
30        }
31        s = ns;
32        for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
33            int c = smalls[j>>1] - pc;
34            for(int e=(j*p)>>1; i>=e; --i) smalls[i] -= c;
35        }
36        ++pc;
37    }
38    larges[0] += 1LL*(s + 2*(pc-1))*(s-1) >> 1;
39    for(int k=1; k<s; ++k) larges[0] -= larges[k];
40    for(int l=1; l<s; ++l){
41        int q = roughs[l];
42        u64 m = n / (u64)q;
43        long long t = 0;
44        int e = smalls[(int)((m/q - 1) >> 1)] - pc;
45        if(e < l+1) break;
46        for(int k=l+1; k<=e; ++k) t += smalls[(int)((m/
47            (u64)roughs[k] - 1) >> 1)];
48        larges[0] += t - 1LL*(e - l)*(pc + l - 1);
49    }

```

```

48 return (u64)(larges[0] + 1);
49 }

```

## 12.17 Linear Sieve for Other Number Theoretic Functions

d9fd0b

```

1 // linear_sieve(n, primes, lp, phi, mu, d, sigma)
2 // Outputs over the index range 0..n (n >= 1):
3 // primes : all primes in [2..n], increasing.
4 // lp      : lowest prime factor; lp[1]=0, lp[x] is the
5 //           smallest prime dividing x.
6 // phi     : Euler totient, phi[x] = |{1<=k<=x :
7 //           gcd(k,x)=1}|. Multiplicative.
8 // mu      : Möbius; mu[1]=1, mu[x]=0 if x has a squared
9 //           prime factor, else (-1)^{#distinct primes}.
10 // d       : number of divisors; if x=∏ p_i^{e_i}, then
11 //           d[x]=∏(e_i+1). Multiplicative.
12 // sigma   : sum of divisors; if x=∏ p_i^{e_i}, then
13 //           sigma[x]=∏(1+p_i+...+p_i^{e_i}). (use ll)
14 // Complexity: O(n) time, O(n) memory.
15 // Notes: Arrays are resized inside; primes is cleared and
16 //         reserved. sigma uses ll to avoid 32-bit overflow.
17 static inline void linear_sieve(
18     int n,
19     std::vector<int> &primes,
20     std::vector<int> &lp,
21     std::vector<int> &phi,
22     std::vector<int> &mu,
23     std::vector<int> &d,
24     std::vector<ll> &sigma
25 ) {
26     lp.assign(n + 1, 0); phi.assign(n + 1, 0); mu.assign(n +
27     1, 0); d.assign(n + 1, 0); sigma.assign(n + 1, 0);
28     primes.clear(); primes.reserve(n > 1 ? n / 10 : 0);
29     std::vector<int> cnt(n + 1, 0), core(n + 1, 1);
30     std::vector<ll> p_pow(n + 1, 1), sum_p(n + 1, 1);
31     phi[1] = mu[1] = d[1] = sigma[1] = 1;
32     for (int i = 2; i <= n; ++i) {
33         if (!lp[i]) {
34             lp[i] = i; primes.push_back(i);
35             phi[i] = i - 1; mu[i] = -1; d[i] = 2;
36             cnt[i] = 1; p_pow[i] = i; core[i] = 1;
37             sum_p[i] = 1 + (ll)i; sigma[i] = sum_p[i];
38         }
39         for (int p : primes) {
40             long long ip = 1LL * i * p;
41             if (ip > n) break;
42             lp[ip] = p;
43             if (p == lp[i]) {
44                 cnt[ip] = cnt[i] + 1; p_pow[ip] = p_pow[i] * p;
45                 core[ip] = core[i];
46                 sum_p[ip] = sum_p[i] + p_pow[ip];
47                 phi[ip] = phi[i] * p; mu[ip] = 0;
48                 d[ip] = d[core[ip]] * (cnt[ip] + 1);
49                 sigma[ip] = sigma[core[ip]] * sum_p[ip];
50                 break; // critical for linear complexity
51             } else {
52                 cnt[ip] = 1; p_pow[ip] = p; core[ip] = i;
53                 sum_p[ip] = 1 + (ll)p;
54                 phi[ip] = phi[i] * (p - 1); mu[ip] = -mu[i];
55                 d[ip] = d[i] * 2;
56                 sigma[ip] = sigma[i] * sum_p[ip];
57             }
58         }
59     }
60 }
61 // Optional helper: factorize x in O(log x) using lp
62 // (requires x in [2..n])
63 static inline std::vector<std::pair<int, int>>
64 factorize(int x, const std::vector<int> &lp) {
65     std::vector<std::pair<int, int>> res;
66     while (x > 1) {
67         int p = lp[x], e = 0;
68         do { x /= p; ++e; } while (x % p == 0);
69         res.push_back({p, e});
70     }
71     return res;
72 }

```

## 13 Numerical

### 13.1 Polynomials and recurrences

```

1 /* Author: David Rydh, Per Austrin
2 * Description: */
3 struct Poly {
4     vector<double> a;
5     double operator()(double x) const {
6         double val = 0;
7         for (int i = sz(a); i--;) (val *= x) += a[i];
8         return val;
9     }

```

```

10 void diff() {
11     rep(i, 1, sz(a)) a[i-1] = i*a[i];
12     a.pop_back();
13 }
14 void divroot(double x0) {
15     double b = a.back(), c; a.back() = 0;
16     for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b,
17     b=c;
18     a.pop_back();
19 }
20
21 /* Author: Per Austrin (License: CC0)
22 * Description: Finds the real roots to a polynomial.
23 * Usage: polyRoots({{2,-3,1}}, -1e9, 1e9) // solve x^2-3x+2 = 0
24 * Time: O(n^2 \log(1/\epsilon)) */
25 #include "Polynomial.cpp"
26 vector<double> polyRoots(Poly p, double xmin, double xmax) {
27     if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
28     vector<double> ret;
29     Poly der = p;
30     der.diff();
31     auto dr = polyRoots(der, xmin, xmax);
32     dr.push_back(xmin-1);
33     dr.push_back(xmax+1);
34     sort(all(dr));
35     rep(i, 0, sz(dr)-1) {
36         double l = dr[i], h = dr[i+1];
37         bool sign = p(l) > 0;
38         if (sign ^ (p(h) > 0)) {
39             rep(it, 0, 60) { // while (h - l > 1e-8)
40                 double m = (l + h) / 2, f = p(m);
41                 if ((f <= 0) ^ sign) l = m;
42                 else h = m;
43             }
44             ret.push_back((l + h) / 2);
45         }
46     }
47     return ret;
48 }
49
50 /* Author: Simon Lindholm (License: CC0)
51 * Description: Given n points (x[i], y[i]), computes an
52   n-1-degree polynomial p(x) that
53   passes through them: p(x) = a[0]*x^0 + ... +
54   a[n-1]*x^{n-1}.
55 * For numerical precision, pick x[k] = c*cos(k/(n-1)*pi),
56   k=0 \dots n-1.
57 * Time: O(n^2) */
58 typedef vector<double> vd;
59 vd interpolate(vd x, vd y, int n) {
60     vd res(n), temp(n);
61     rep(k, 0, n-1) rep(i, k+1, n)
62         y[i] = (y[i] - y[k]) / (x[i] - x[k]);
63     double last = 0; temp[0] = 1;
64     rep(k, 0, n) rep(i, 0, n) {
65         res[i] += y[k] * temp[i];
66         swap(last, temp[i]);
67         temp[i] -= last * x[k];
68     }
69     return res;
70 }
71
72 /* Author: koukanni (License: CC0)
73 * Description: Lagrange interpolation for polynomial
74   evaluation at integer points.
75 * Given polynomial values f[0], f[1], ..., f[n-1], computes
76   f(c).
77 * Time: O(n)
78 * Status: tested */
79 Mint lagrange_iota(const vector<Mint> &f, const int c) {
80     const int n = int(size(f));
81     if (c < n) {
82         return f[c];
83     }
84     vector<Mint> fac(n), ifac(n);
85     fac[0] = 1;
86     for (int i = 1; i < n; i++) fac[i] = fac[i-1] * i;
87     ifac[n-1] = fac[n-1].inv();
88     for (int i = n-1; i >= 1; i--) ifac[i-1] = ifac[i] *
89     i;
90     vector<Mint> g(n);
91     for (int i = 0; i < n; i++) {
92         g[i] = f[i] * ifac[i] * ifac[n-1-i];
93         if ((n-1-i) & 1) g[i] *= -1;
94     }
95     vector<Mint> lp(n+1), rp(n+1);
96     lp[0] = rp[n] = 1;

```

```

23     for (int i = 0; i < n; i++) lp[i+1] = lp[i] * Mint(c -
24     i);
25     for (int i = n-1; i >= 0; i--) rp[i] = rp[i+1] *
26     Mint(c - i);
27     Mint ans = 0;
28     for (int i = 0; i < n; i++) ans += g[i] * lp[i] * rp[i +
29     1];
30     return ans;
31 }
32
33 /* Author: Lucian Bicsi (License: CC0)
34 * Description: Recovers any n-order linear recurrence
35   relation from the first
36   2n terms of the recurrence.
37 * Useful for guessing linear recurrences after brute-forcing
38   the first terms.
39 * Should work on any field, but numerical stability for
40   floats is not guaranteed.
41 * Output will have size O(n).
42 * Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
43 * Time: O(N^2), Status: brute-force-tested mod 5 for n <= 5
44   and all s */
45 #include "../number-theory/ModPow.cpp"
46 vector<ll> berlekampMassey(vector<ll> s) {
47     int n = sz(s), L = 0, m = 0;
48     vector<ll> C(n), B(n), T;
49     C[0] = B[0] = 1;
50     ll b = 1;
51     rep(i, 0, n) { ++m;
52         ll d = s[i] % mod;
53         rep(j, L+1, i) d = (d + C[j] * s[i-j]) % mod;
54         if (!d) continue;
55         T = C; ll coef = d * modpow(b, mod-2) % mod;
56         rep(j, m, n) C[j] = (C[j] - coef * B[j-m]) % mod;
57         if (2 * L > i) continue;
58         L = i + 1 - L; B = T; b = d; m = 0;
59     }
60     // assert(2 * L + 30 <= n);
61     C.resize(L+1); C.erase(C.begin());
62     for (ll& x : C) x = (mod - x) % mod;
63     return C;
64 }
65
66 /* Author: Lucian Bicsi (License: CC0)
67 * Description: Generates the k'th term of an n-order
68   linear recurrence S[i] = \sum_j S[i-j-1]tr[j]$,
69   given S[0] \dots S[n-1] and tr[0] \dots tr[n-1]$.
70 * Faster than matrix multiplication.
71 * Useful together with Berlekamp-Massey.
72 * Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci
73   number
74 * Time: O(n^2 \log k)
75 * Status: brute-force-tested mod 5 for n <= 5 */
76 const ll mod = 5; /** exclude-line */
77 typedef vector<ll> Poly;
78 ll linearRec(Poly S, Poly tr, ll k) {
79     int n = sz(tr);
80     auto combine = [&](Poly a, Poly b) {
81         Poly res(n+2+1);
82         rep(i, 0, n+1) rep(j, 0, n+1)
83             res[i+j] = (res[i+j] + a[i] * b[j]) % mod;
84         for (int i = 2 * n; i > n; --i) rep(j, 0, n)
85             res[i-1-j] = (res[i-1-j] + res[i] * tr[j]) %
86             mod;
87         res.resize(n+1);
88         return res;
89     };
90     Poly pol(n+1, e(pol));
91     pol[0] = e[1] = 1;
92     for (++k; k; k /= 2) {
93         if (k % 2) pol = combine(pol, e);
94         e = combine(e, e);
95     }
96     ll res = 0;
97     rep(i, 0, n) res = (res + pol[i+1] * S[i]) % mod;
98     return res;
99 }

```

## 13.2 Optimization

```

1 /* Author: Ulf Lundstrom (License: CC0)
2 * Description: Finds the argument minimizing the function
3   f(x) in the interval [a,b]
4 * assuming f(x) is unimodal on the interval, i.e. has only
5   one local minimum and no local
6   maximum. The maximum error in the result is eps$. Works
7   equally well for maximization
8 * with a small change in the code. See TernarySearch.h in
9   the Various chapter for a
10 * discrete version.
11 * Usage: double func(double x) { return 4*x+.3*x*x; }
12 * double xmin = gss(-1000, 1000, func);
13 * Time: O(\log((b-a) / \epsilon)), Status: tested */

```

```

10 /// It is important for r to be precise, otherwise we don't
   necessarily maintain the inequality  $a < x_1 < x_2 < b$ .
11 double gss(double a, double b, double (*f)(double)) {
12     double r = (sqrt(5)-1)/2, eps = 1e-7;
13     double x1 = b - r*(b-a), x2 = a + r*(b-a);
14     double f1 = f(x1), f2 = f(x2);
15     while (b-a > eps)
16     if (f1 < f2) { //change to > to find maximum
17         b = x2; x2 = x1; f2 = f1;
18         x1 = b - r*(b-a); f1 = f(x1);
19     } else {
20         a = x1; x1 = x2; f1 = f2;
21         x2 = a + r*(b-a); f2 = f(x2);
22     }
23     return a;
24 }

1/* Author: Simon Lindholm (License: CC0)
2* Description: Poor man's optimization for unimodal
   functions.
3* Status: used with great success */
4 typedef array<double, 2> P;
5 template<class F> pair<double, P> hillClimb(P start, F f) {
6     pair<double, P> cur(f(start), start);
7     for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
8         rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
9             P p = cur.second;
10            p[0] += dx*jmp;
11            p[1] += dy*jmp;
12            cur = min(cur, make_pair(f(p), p));
13        }
14    }
15    return cur;
16 }

1/* Author: Simon Lindholm (License: CC0)
2* Description: Simple integration of a function over an
   interval using
3* Simpson's rule. The error should be proportional to  $h^4$ ,
   although in
4* practice you will want to verify that the result is stable
   to desired
5* precision when epsilon changes.
6* Status: mostly untested */
7 template<class F>
8 double quad(double a, double b, F f, const int n = 1000) {
9     double h = (b - a) / 2 / n, v = f(a) + f(b);
10    rep(i, 1, n*2)
11        v += f(a + i*h) * (i&1 ? 4 : 2);
12    return v * h / 3;
13 }

1/* Author: Simon Lindholm (License: CC0)
2* Description: Fast integration using an adaptive Simpson's
   rule.
3* Usage:
4* double sphereVolume = quad(-1, 1, [](double x) {
5*     return quad(-1, 1, [&](double y) {
6*         return quad(-1, 1, [&](double z) {
7*             return x*x + y*y + z*z < 1; });});});
8* Status: mostly untested */
9 typedef double d;
10 #define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
11 template <class F>
12 d rec(F& f, d a, d b, d eps, d S) {
13     d c = (a + b) / 2;
14     d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
15     if (abs(T - S) <= 15 * eps || b - a < 1e-10)
16         return T + (T - S) / 15;
17     return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
18     S2);
19 }
20 template<class F>
21 d quad(d a, d b, F f, d eps = 1e-8) {
22     return rec(f, a, b, eps, S(a, b));
23 }

1/* Author: Stanford (License: MIT)
2* Description: Solves a general linear maximization problem:
   maximize  $\sum c^T x$  subject to  $Ax \leq b$ ,  $x \geq 0$ .
3* Returns -inf if there is no solution, inf if there are
   arbitrarily good solutions, or the maximum value of  $\sum c^T x$ 
   otherwise.
4* The input vector is set to an optimal  $x$  (or in the
   unbounded case, an arbitrary solution fulfilling the
   constraints).
5* Numerical stability is not guaranteed. For better
   performance, define variables such that  $x = 0$  is viable.
6* Usage:
7* vvd A = {{1,-1}, {-1,1}, {-1,-2}};
8* vd b = {1,1,-4}, c = {-1,-1}, x;

```

```

9* T val = LPSolver(A, b, c).solve(x);
10* Time:  $O(NM * \#\text{pivots})$ , where a pivot may be e.g. an edge
   relaxation.  $O(2^n)$  in the general case.
11* Status: seems to work? */
12 typedef double T; // long double, Rational, double +
   mod<P>...
13 typedef vector<T> vd;
14 typedef vector<vd> vvd;
15 const T eps = 1e-8, inf = 1/.0;
16 #define MP make_pair
17 #define ltj(X) if(s == -1 || MP(X[j], N[j]) < MP(X[s], N[s]))
   s=j
18 struct LPSolver {
19     int m, n; vi N, B; vvd D;
20     LPSolver(const vvd& A, const vd& b, const vd& c) :
21         m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
22         rep(i, 0, m) rep(j, 0, n) D[i][j] = A[i][j];
23         rep(i, 0, m) { B[i] = n+1; D[i][n] = -1; D[i][n+1] =
24         b[i]; }
25         rep(j, 0, n) { N[j] = j; D[m][j] = -c[j]; }
26         N[n] = -1; D[m+1][n] = 1;
27     }
28     void pivot(int r, int s) {
29         T *a = D[r].data(), inv = 1 / a[s];
30         rep(i, 0, m+2) if (i != r && abs(D[i][s]) > eps) {
31             T *b = D[i].data(), inv2 = b[s] * inv;
32             rep(j, 0, n+2) b[j] -= a[j] * inv2;
33             b[s] = a[s] * inv2;
34         }
35         rep(j, 0, n+2) if (j != s) D[r][j] *= inv;
36         rep(i, 0, m+2) if (i != r) D[i][s] *= -inv;
37         D[r][s] = inv;
38         swap(B[r], N[s]);
39     }
40     bool simplex(int phase) {
41         int x = m + phase - 1;
42         for (;;) {
43             int s = -1;
44             rep(j, 0, n+1) if (N[j] != -phase) ltj(D[x]);
45             if (D[x][s] >= -eps) return true;
46             int r = -1;
47             rep(i, 0, m) {
48                 if (D[i][s] <= eps) continue;
49                 if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
50                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
51             }
52             if (r == -1) return false;
53             pivot(r, s);
54         }
55     }
56     T solve(vd &x) {
57         int r = 0;
58         rep(i, 1, m) if (D[i][n+1] < D[r][n+1]) r = i;
59         if (D[r][n+1] < -eps) {
60             pivot(r, n);
61             if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
62             rep(i, 0, m) if (B[i] == -1) {
63                 int s = 0;
64                 rep(j, 1, n+1) ltj(D[i]);
65                 pivot(i, s);
66             }
67         }
68         bool ok = simplex(1); x = vd(n);
69         rep(i, 0, m) if (B[i] < n) x[B[i]] = D[i][n+1];
70         return ok ? D[m][n+1] : inf;
71     }
72 }

```

### 13.3 Matrices

```

1/* Author: Simon Lindholm (License: CC0)
2* Description: Calculates determinant of a matrix. Destroys
   the matrix.
3* Time:  $O(N^3)$ , Status: somewhat tested */
4 double det(vector<vector<double>>& a) {
5     int n = sz(a); double res = 1;
6     rep(i, 0, n) {
7         int b = i;
8         rep(j, i+1, n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
9         if (i != b) swap(a[i], a[b]), res *= -1;
10        res *= a[i][i];
11        if (res == 0) return 0;
12        rep(j, i+1, n) {
13            double v = a[j][i] / a[i][i];
14            if (v != 0) rep(k, i+1, n) a[j][k] -= v * a[i][k];
15        }
16    }
17    return res;
18 }

1/* Author: Unknown

```

```

2 * Description: Calculates determinant using modular
  arithmetics.
3 * Modulos can also be removed to get a pure-integer version.
4 * Time:  $O(N^3)$ 
5 * Status: brute-force-tested for  $N \leq 3$ ,  $\text{mod} \leq 7$  */
6 const ll mod = 12345;
7 ll det(vector<vector<ll>>& a) {
8     int n = sz(a); ll ans = 1;
9     rep(i, 0, n) {
10         rep(j, i+1, n) {
11             while (a[j][i] != 0) { // gcd step
12                 ll t = a[i][i] / a[j][i];
13                 if (t) rep(k, i, n)
14                     a[i][k] = (a[i][k] - a[j][k] * t) % mod;
15                 swap(a[i], a[j]);
16                 ans *= -1;
17             }
18         }
19         ans = ans * a[i][i] % mod;
20         if (!ans) return 0;
21     }
22     return (ans + mod) % mod;
23 }

1 /* Author: Per Austrin, Simon Lindholm (License: CC0)
2 * Description: Solves  $A * x = b$ . If there are multiple
  solutions, an arbitrary one is returned.
3 * Returns rank, or -1 if no solutions. Data in  $A$  and  $b$ 
  is lost.
4 * Time:  $O(n^2 m)$ 
5 * Status: tested on kattis:equationsolver, and brute-force-
  tested mod 3 and 5 for  $n, m \leq 3$  */
6 typedef vector<double> vd;
7 const double eps = 1e-12;
8 int solveLinear(vector<vd>& A, vd& b, vd& x) {
9     int n = sz(A), m = sz(x), rank = 0, br, bc;
10    if (n) assert(sz(A[0]) == m);
11    vi col(m); iota(all(col), 0);
12    rep(i, 0, n) {
13        double v, bv = 0;
14        rep(r, i, n) rep(c, i, m)
15            if ((v = fabs(A[r][c])) > bv)
16                br = r, bc = c, bv = v;
17        if (bv <= eps) {
18            rep(j, i, n) if (fabs(b[j]) > eps) return -1;
19            break;
20        }
21        swap(A[i], A[br]);
22        swap(b[i], b[br]);
23        swap(col[i], col[bc]);
24        rep(j, 0, n) swap(A[j][i], A[j][bc]);
25        bv = 1/A[i][i];
26        rep(j, i+1, n) {
27            double fac = A[j][i] * bv;
28            b[j] -= fac * b[i];
29            rep(k, i+1, m) A[j][k] -= fac * A[i][k];
30        }
31        rank++;
32    }
33    x.assign(m, 0);
34    for (int i = rank; i--;) {
35        b[i] /= A[i][i];
36        x[col[i]] = b[i];
37        rep(j, 0, i) b[j] -= A[j][i] * b[i];
38    }
39    return rank; // (multiple solutions if rank < m)
40 }

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: To get all uniquely determined values of  $x$ 
  back from SolveLinear, make the following changes:
3 * Status: tested on kattis:equationsolverplus, stress-tested
  */
4 #include "SolveLinear.cpp"
5 rep(j, 0, n) if (j != i) // instead of rep(j, i+1, n)
6 // ... then at the end:
7 x.assign(m, undefined);
8 rep(i, 0, rank) {
9     rep(j, rank, m) if (fabs(A[i][j]) > eps) goto fail;
10    x[col[i]] = b[i] / A[i][i];
11 fail; }

1 /* Author: Simon Lindholm (License: CC0)
2 * Description: Solves  $Ax = b$  over  $\mathbb{F}_2$ . If there
  are multiple solutions, one is returned arbitrarily.
3 * Returns rank, or -1 if no solutions. Destroys  $A$  and  $b$ .
4 * Time:  $O(n^2 m)$ 
5 * Status: brute-force-tested for  $n, m \leq 4$  */
6 typedef bitset<1000> bs;
7 int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
8     int n = sz(A), rank = 0, br;
9     assert(m <= sz(x));
10    vi col(m); iota(all(col), 0);
11    rep(i, 0, n) {
12        for (br=i; br<n; ++br) if (A[br].any()) break;
13        if (br == n) {
14            rep(j, i, n) if (b[j]) return -1;
15            break;
16        }
17        int bc = (int)A[br]._Find_next(i-1);
18        swap(A[i], A[br]);
19        swap(b[i], b[br]);
20        swap(col[i], col[bc]);
21        rep(j, 0, n) if (A[j][i] != A[j][bc]) {
22            A[j].flip(i); A[j].flip(bc);
23        }
24        rep(j, i+1, n) if (A[j][i]) {
25            b[j] ^= b[i];
26            A[j] ^= A[i];
27        }
28        rank++;
29    }
30    x = bs();
31    for (int i = rank; i--;) {
32        if (!b[i]) continue;
33        x[col[i]] = 1;
34        rep(j, 0, i) b[j] ^= A[j][i];
35    }
36    return rank; // (multiple solutions if rank < m)
37 }

1 /* Author: Max Bennedich
2 * Description: Invert matrix  $A$ . Returns rank; result is
  stored in  $A$  unless singular (rank < n).
3 * Can easily be extended to prime moduli; for prime powers,
  repeatedly
4 * set  $A^{-1} = A^{-1} (2I - AA^{-1}) \pmod{p^k}$ 
  where  $A^{-1}$  starts as
5 * the inverse of  $A \pmod{p}$ , and  $k$  is doubled in each step.
6 * Time:  $O(n^3)$ 
7 * Status: Slightly tested */
8 int matInv(vector<vector<double>>& A) {
9     int n = sz(A); vi col(n);
10    vector<vector<double>> tmp(n, vector<double>(n));
11    rep(i, 0, n) tmp[i][i] = 1, col[i] = i;
12    rep(i, 0, n) {
13        int r = i, c = i;
14        rep(j, i, n) rep(k, i, n)
15            if (fabs(A[j][k]) > fabs(A[r][c]))
16                r = j, c = k;
17        if (fabs(A[r][c]) < 1e-12) return i;
18        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
19        rep(j, 0, n)
20            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
21        swap(col[i], col[c]);
22        double v = A[i][i];
23        rep(j, i+1, n) {
24            double f = A[j][i] / v;
25            A[j][i] = 0;
26            rep(k, i+1, n) A[j][k] -= f * A[i][k];
27            rep(k, 0, n) tmp[j][k] -= f * tmp[i][k];
28        }
29        rep(j, i+1, n) A[i][j] /= v;
30        rep(j, 0, n) tmp[i][j] /= v;
31        A[i][i] = 1;
32    }
33    // forget A at this point, just eliminate tmp backward
34    for (int i = n-1; i > 0; --i) rep(j, 0, i) {
35        double v = A[j][i];
36        rep(k, 0, n) tmp[j][k] -= v * tmp[i][k];
37    }
38    rep(i, 0, n) rep(j, 0, n) A[col[i]][col[j]] = tmp[i][j];
39    return n;
40 }

1 /* Author: Simon Lindholm
2 * Description: Invert matrix  $A$  modulo a prime.
3 * Returns rank; result is stored in  $A$  unless singular
  (rank < n).
4 * For prime powers, repeatedly set  $A^{-1} = A^{-1} (2I -
  AA^{-1}) \pmod{p^k}$  where  $A^{-1}$  starts as
5 * the inverse of  $A \pmod{p}$ , and  $k$  is doubled in each step.
6 * Time:  $O(n^3)$ 
7 * Status: Slightly tested */
8 #include "../number-theory/ModPow.cpp"
9 int matInv(vector<vector<ll>>& A) {
10    int n = sz(A); vi col(n);
11    vector<vector<ll>> tmp(n, vector<ll>(n));
12    rep(i, 0, n) tmp[i][i] = 1, col[i] = i;
13    rep(i, 0, n) {

```

```

14     int r = i, c = i;
15     rep(j,i,n) rep(k,i,n) if (A[j][k]) {
16         r = j; c = k; goto found;
17     }
18     return i;
19 found:
20     A[i].swap(A[r]); tmp[i].swap(tmp[r]);
21     rep(j,0,n)
22         swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
23     swap(col[i], col[c]);
24     ll v = modpow(A[i][i], mod - 2);
25     rep(j,i+1,n) {
26         ll f = A[j][i] * v % mod;
27         A[j][i] = 0;
28         rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
29         rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) %
mod;
30     }
31     rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
32     rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
33     A[i][i] = 1;
34 }
35 for (int i = n-1; i > 0; --i) rep(j,0,i) {
36     ll v = A[j][i];
37     rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
38 }
39 rep(i,0,n) rep(j,0,n)
40     A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] <
0)*mod;
41 return n;
42 }

1/** Author: Ulf Lundstrom, Simon Lindholm (License: CC0)
2 * Description: Solves the tridiagonal matrix equation
system:
3 * [ b_0 ] [ d_0 p_0 0 0 ... 0 ] [ x_0 ]
4 * [ b_1 ] [ q_0 d_1 p_1 0 ... 0 ] [ x_1 ]
5 * [ b_2 ] [ 0 q_1 d_2 p_2 ... 0 ] [ x_2 ]
6 * [ ... ] [ ... ... ... ... ] [ ... ]
7 * [ b_{n-1} ] [ 0 0 ... 0 q_{n-2} d_{n-1} ] [ x_{n-1} ]
8 * Application:
9 * Solves linear recurrence systems of the form:
10 * a_i = b_i * a_{i-1} + c_i * a_{i+1} + d_i (1 <= i <=
n)
11 * where a_0, a_{n+1}, b_i, c_i, and d_i are given.
12 * The solution vector a can be obtained via:
13 * d = { 1, -1, -1, ..., -1, 1 }
14 * p = { 0, c_1, c_2, ..., c_n }
15 * q = { b_1, b_2, ..., b_n, 0 }
16 * b = { a_0, d_1, d_2, ..., d_n, a_{n+1} }
17 * Numerical Stability:
18 * - Fails if the solution is not unique.
19 * - Numerically stable without pivot checks if:
20 * 1) Strictly diagonally dominant:
21 * |d_i| > |p_i| + |q_{i-1}| for all i, OR
22 * |d_i| > |p_{i-1}| + |q_i| for all i
23 * 2) The matrix is symmetric positive-definite.
24 * Complexity: O(N) time, O(N) space. Status: Brute-force
tested mod 5 and 7, stress-tested for real matrices. */
25 typedef double T;
26 vector<T> tridiagonal(vector<T> diag, const vector<T>&
super,
27 const vector<T>& sub, vector<T> b) {
28     int n = sz(b); vi tr(n);
29     rep(i,0,n-1) {
30         if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] ==
0
31             b[i+1] -= b[i] * diag[i+1] / super[i];
32             if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
33             diag[i+1] = sub[i]; tr[i+1] = 1;
34         } else {
35             diag[i+1] -= super[i]*sub[i]/diag[i];
36             b[i+1] -= b[i]*sub[i]/diag[i];
37         }
38     }
39     for (int i = n; i--;) {
40         if (tr[i]) {
41             swap(b[i], b[i-1]);
42             diag[i-1] = diag[i];
43             b[i] /= super[i-1];
44         } else {
45             b[i] /= diag[i];
46             if (i) b[i-1] -= b[i]*super[i-1];
47         }
48     }
49     return b;
50 }

2 * Author: chilli, Ludo Pulles, Simon Lindholm (License:
CC0)
3 * Description: Higher precision FFT, can be used for
convolutions modulo arbitrary integers
4 * as long as N\log_2N * mod < 8.6e14 (in practice 10^16 or
higher).
5 * Inputs must be in [0, mod).
6 * Time: O(N \log N), where N = |A| + |B| (twice as slow as
NTT or normal FFT), Status: stress-tested */
7 typedef complex<double> C;
8 typedef vector<ll> vl;
9 void fft(vector<C>& a) {
10     int n = sz(a), L = 31 - __builtin_clz(n);
11     static vector<complex<long double>> R(2, 1);
12     static vector<C> rt(2, 1);
13     for (static int k = 2; k < n; k *= 2) {
14         R.resize(n); rt.resize(n);
15         auto x = polar(1.0L, acos(-1.0L) / k);
16         rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x :
R[i / 2];
17     }
18     vi rev(n);
19     rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
20     rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
21     for (int k = 1; k < n; k *= 2)
22         for (int i = 0; i < n; i += 2 * k) rep(j, 0, k) {
23             auto x = (double *)&rt[j + k], y = (double *)&a[i + j
+ k];
24             C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] *
y[0]);
25             a[i + j + k] = a[i + j] - z;
26             a[i + j] += z;
27         }
28 }
29 template<int M> vl convMod(const vl &a, const vl &b) {
30     if (a.empty() || b.empty()) return {};
31     vl res(sz(a) + sz(b) - 1);
32     int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut =
int(sqrt(M));
33     vector<C> L(n), R(n), outs(n), outl(n);
34     rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] %
cut);
35     rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] %
cut);
36     fft(L), fft(R);
37     rep(i, 0, n) {
38         int j = -i & (n - 1);
39         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
40         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
41     }
42     fft(outl), fft(outs);
43     rep(i, 0, sz(res)) {
44         ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i])
+ .5);
45         ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
46         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
47     }
48     return res;
49 }

1/** Author: Lucian Bicsi (License: GNU Free Documentation
License 1.2)
2 * Description: Transform to a basis with fast convolutions
of the form
3 * c[z] = \sum_z = x \oplus y a[x] * b[y]
4 * where \oplus is one of AND, OR, XOR.
5 * The size of a$ must be a power of 2.
6 * Time: O(N \log N), Status: stress-tested */
7 void FST(vi& a, bool inv) {
8     for (int n = sz(a), step = 1; step < n; step *= 2) {
9         for (int i = 0; i < n; i += 2 * step) rep(j, i, i+step) {
10             int &u = a[j], &v = a[j + step]; tie(u, v) =
11                 inv ? pii(v - u, u) : pii(v, u + v); // AND
12                 // inv ? pii(v, u - v) : pii(u + v, u); // OR ///
include-line
13                 // pii(u + v, u - v); // XOR ///
include-line
14         }
15     }
16     // if (inv) for (int& x : a) x /= sz(a); // XOR only ///
include-line
17 }
18 vi conv(vi a, vi b) {
19     FST(a, 0); FST(b, 0);
20     rep(i,0,sz(a)) a[i] *= b[i];
21     FST(a, 1); return a;
22 }

```

## 13.4 Fourier transforms

1/\*\*

## 14 Combinatorics

### 14.1 Catalan Number

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, C_n = C_n^{2n} - C_{n-1}^{2n}$$

|    |        |        |         |         |
|----|--------|--------|---------|---------|
| 0  | 1      | 1      | 2       | 5       |
| 4  | 14     | 42     | 132     | 429     |
| 8  | 1430   | 4862   | 16796   | 58786   |
| 12 | 208012 | 742900 | 2674440 | 9694845 |

### 14.2 Bertrand's Ballot Theorem

- $A$  always  $> B$ :  $C(p+q, p) - 2C(p+q-1, p)$
- $A$  always  $\geq B$ :  $C(p+q, p) \times \frac{p+1-q}{p+1}$

### 14.3 Burnside's Lemma

Let  $X$  be the original set.

Let  $G$  be the group of operations acting on  $X$ .

Let  $X^g$  be the set of  $x$  not affected by  $g$ .

Let  $X/G$  be the set of orbits. Then the following equation holds:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

## 15 Special Numbers

### 15.1 Fibonacci Series

|    |         |         |         |          |
|----|---------|---------|---------|----------|
| 1  | 1       | 1       | 2       | 3        |
| 5  | 5       | 8       | 13      | 21       |
| 9  | 34      | 55      | 89      | 144      |
| 13 | 233     | 377     | 610     | 987      |
| 17 | 1597    | 2584    | 4181    | 6765     |
| 21 | 10946   | 17711   | 28657   | 46368    |
| 25 | 75025   | 121393  | 196418  | 317811   |
| 29 | 514229  | 832040  | 1346269 | 2178309  |
| 33 | 3524578 | 5702887 | 9227465 | 14930352 |

$$f(45) \approx 10^9, f(88) \approx 10^{18}$$

### 15.2 Prime Numbers

- $\pi(n) \equiv$  Number of primes  $\leq n \approx \frac{n}{(\ln n)-1}$
- $\pi(100) = 25, \pi(200) = 46$
- $\pi(500) = 95, \pi(1000) = 168$
- $\pi(2000) = 303, \pi(4000) = 550$
- $\pi(10^4) = 1229, \pi(10^5) = 9592$
- $\pi(10^6) = 78498, \pi(10^7) = 664579$

### 15.3 Number of Divisors

- If  $n = \prod p_i^{a_i}$ , then  $\tau(n) = \prod (a_i + 1)$
- Maximum  $\tau(n)$  for  $n \leq 10^k$ :

| $k$ | $\max \tau(n)$ | $k$ | $\max \tau(n)$ | $k$ | $\max \tau(n)$ |
|-----|----------------|-----|----------------|-----|----------------|
| 3   | 32             | 9   | 1344           | 15  | 26880          |
| 4   | 64             | 10  | 2304           | 16  | 41472          |
| 5   | 128            | 11  | 4032           | 17  | 64512          |
| 6   | 240            | 12  | 6720           | 18  | 103680         |
| 7   | 448            | 13  | 10752          |     |                |
| 8   | 768            | 14  | 17280          |     |                |

- Useful bounds:  $n \leq 10^6 \Rightarrow \tau(n) \leq 240$
- $n \leq 10^9 \Rightarrow \tau(n) \leq 1344$
- $n \leq 10^{12} \Rightarrow \tau(n) \leq 6720$
- $n \leq 10^{15} \Rightarrow \tau(n) \leq 26880$
- $n \leq 10^{18} \Rightarrow \tau(n) \leq 103680$

### 15.4 Distinct Prime Factors

- Let  $\omega(n)$  = number of distinct prime factors.
- Minimum number with  $k$  distinct prime factors is the product of the first  $k$  primes.
  - $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 = 9699690$
  - $2 \cdot 3 \cdot \dots \cdot 23 = 223092870$
  - $2 \cdot 3 \cdot \dots \cdot 29 = 6469693230$
  - $n \leq 10^9 \Rightarrow \omega(n) \leq 9, n \leq 10^{18} \Rightarrow \omega(n) \leq 15$
- Number of square-free divisors =  $2^{\omega(n)}$ 
  - $n \leq 10^9 \Rightarrow \leq 512, n \leq 10^{18} \Rightarrow \leq 32768$